IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 7 May 2026, accepted 31 May 2026, date of publication 4 June 2026, date of current version 11 June 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3700492

TOPICAL REVIEW

A Systematic Literature Review of Prompt Injection Attacks in LLM-Integrated Systems

CHRISTIAN WOESLE, (Member, IEEE), AND RICARDO BUETTNER, (Senior Member, IEEE)
Chair of Hybrid Intelligence, Helmut-Schmidt-University/University of the Federal Armed Forces Hamburg, 22043 Hamburg, Germany

Corresponding author: Ricardo Buettner (buettner@hsu-hh.de)

This work was supported by the Open-Access-Publication-Fund of the Helmut-Schmidt-University/University of the Federal Armed Forces Hamburg.

ABSTRACT Prompt injection exploits a fundamental property of large language models: the inability to distinguish trusted instructions from untrusted content when both are processed as natural language tokens. As these models are integrated into tool-augmented and memory-enabled systems, the attack surface expands beyond the prompt interface, yet no existing review maps this landscape against standardised adversarial frameworks or analyses the balance between offensive and defensive research. This review follows the Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) guidelines. We searched IEEE Xplore, ACM Digital Library, and ScienceDirect for peer-reviewed studies published between 2015 and October 2025, yielding 89 included publications of which 81 form the analytical corpus. Aligned to the MITRE Adversarial Threat Landscape for AI Systems framework (version 5.4.0), each paper was coded along three axes: trust boundary, attacker objective, and research orientation with attacker objectives further classified into seventeen operational patterns aligned to MITRE ATLAS techniques. The corpus is dominated by research on the prompt interface and by two attacker objectives, Instruction Hijack and Policy Bypass, leaving underexplored boundaries lacking both attack characterization and defence proposals. This architectural narrowness does not reflect the multi-component architecture of production LLM systems. The three-axis framework and coded corpus establish a baseline for tracking how coverage evolves as deployment architectures expand. The concentration reflects a research incentive structure that favours reproducible prompt-level experiments over architecturally complex assessments at tool and memory boundaries. The field exhibits a maturity gradient in which defence development is concentrated at the prompt interface while other boundaries remain in early or absent characterization. The corpus treats prompt injection as a model-level property rather than a system-level vulnerability.

INDEX TERMS Prompt injection, large language models, LLM security, trust boundaries, adversarial attacks, MITRE ATLAS, jailbreaking, attack-defence analysis, retrieval-augmented generation, systematic literature review.

I. INTRODUCTION

Large Language Models (LLMs) such as GPT-5, Claude, and Gemini have achieved strong performance across natural language understanding, code generation, and autonomous decision-making [1], [2]. These models learn statistical patterns from massive textual corpora during pretraining, enabling fluent and contextually appropriate language generation [3]. However, as LLM deployment has expanded from research prototypes to production infrastructure in domains such as healthcare [4], finance [5], and cybersecurity [6], a critical security vulnerability has emerged: prompt injection.

Prompt injection refers to the adversarial manipulation of an LLM's instruction stream through crafted inputs at inference time, allowing attackers to overwrite system commands, exfiltrate confidential data, trigger unauthorised actions, or bypass security measures, all without access to

The associate editor coordinating the review of this manuscript and approving it for publication was Tyson Brooks.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

model parameters [7], [8]. This vulnerability is especially relevant for modern LLM systems that function as tool-assisted agents, simultaneously receiving instructions and data from multiple sources: user commands, retrieved documents, tool outputs, and persistent storage. Each of these sources represents a separate trust boundary where adversarial content can enter the model's decision path [8], [9]. The severity of this threat has been demonstrated empirically. In oncology, subvisual prompts embedded in medical image data can suppress the detection of malignant lesions, with attack success rates reaching 67% [10]. Adversarial prompt and fine-tuning attacks on medical LLMs can alter vaccination recommendations and generate harmful medication guidance, with success rates exceeding 80% [11].

Despite innovations in input filtering, structured prompt formatting, and output validation [12], [13], prompt injection remains an open problem. Current mitigation strategies vary in effectiveness depending on architecture and threat model, and there is limited consensus on which solutions generalise [14]. The fragmentation of attack taxonomies, inconsistent terminology (prompt injection, indirect injection, jailbreak, tool exploitation), and the absence of a uniform analytical framework make it difficult to understand the relationships between attack surfaces, attacker objectives, and defence approaches [8], [15].

Although several survey and review efforts address prompt injection and LLM security (see Table 1), covering attack taxonomies, defence catalogues, and adversarial robustness, these efforts remain limited in scope or depth. None attempt a unified synthesis across architectural and operational dimensions. Most existing reviews employ one-dimensional taxonomies that classify work by attack type or defence mechanism in isolation, without linking architectural exposure points to attacker objectives [9]. Critically, existing reviews have not mapped the literature against standardised adversarial frameworks such as MITRE ATLAS [16], creating a disconnect between academic research and the operational threat models used by security practitioners.

To address these gaps, we aim to answer the following research questions (RQ):

- **RQ1:** How is current research on prompt injection distributed across the attack surfaces of LLM-integrated system architectures?
- **RQ2:** Which operational objectives of attackers, as defined in the MITRE ATLAS framework, dominate research in the field of prompt injection, and which remain underrepresented?
- **RQ3:** How is the balance between attack-oriented and defence-oriented research distributed across trust boundaries and attacker objectives, and how has this balance evolved over the publication period?

To answer these questions, we conducted a systematic literature review (SLR) of 89 peer-reviewed publications on LLM prompt injection, carried out in accordance with the PRISMA guidelines [17]. We introduce a three-axis analytical framework that codes each paper along three complementary dimensions: (1) trust boundary, identifying the inference-time architectural interface at which adversarial content first enters the model's decision pathway; (2) ATLAS attacker goal, mapping each paper's attack objectives to five categories derived from the MITRE ATLAS framework (v5.4.0); and (3) research orientation, classifying whether each paper primarily contributes an attack, a defence, or a conceptual/framework-level contribution.

The analysis reveals clear concentrations and critical gaps. The corpus focuses heavily on the prompt interface (TB1 - 83.9%) and is dominated by two attacker objectives, Instruction Hijack (58.0%) and Policy Bypass (29.6%), which together account for 87.7% of the primary targets. Not a single paper in the corpus addresses attacks on tool/function boundaries, and only one addresses persistent memory boundary, despite the widespread deployment of tool-augmented and memory-capable agents. The research orientation analysis shows that the corpus skews towards defence (56.8%), with attack-oriented work at 33.3% and conceptual contributions at 9.9%, indicating that the field has moved beyond initial attack characterisation into active mitigation development (Figure 1).

In light of these findings, the main contributions of this work are:

1) A three-axis analytical framework combining trust boundaries, MITRE ATLAS attacker goals, and research orientation for multidimensional characterisation of prompt injection research.
2) An empirical baseline of 81 papers coded across all three axes, providing a structured, multi-dimensional corpus of prompt injection research with reproducible classification criteria for trust boundaries, ATLAS attacker goals, and attack-defence orientation.
3) A temporal analysis of attack-versus-defence research distribution across trust boundaries and attacker objectives, revealing how the field's investigative focus has shifted from initial attack characterisation towards defence development.

This work is structured as follows: In Section II, preliminary concepts and fundamentals are elaborated to give a brief introduction to the topic. Section III describes our SLR methodology. The results from our SLR are then presented in Section IV and discussed in Section V. Finally, we conclude in Section VI by highlighting the theoretical and practical implications of our findings.

II. PRELIMINARY CONCEPTS AND FUNDAMENTALS

Prompt injection in LLMs refers to adversarial techniques that manipulate the model's command flow at inference time by inserting, overwriting, or subverting the intended commands with crafted inputs [7], [8]. Unlike traditional software vulnerabilities that exploit memory corruption or logic errors in deterministic code, prompt injection exploits a fundamental architectural property of LLMs: the inability to reliably distinguish between trusted instructions from system operators and untrusted content from external sources when

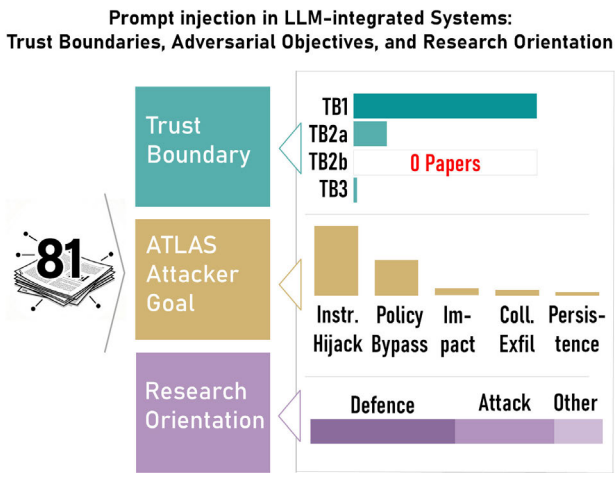


FIGURE 1. 81 papers coded on three axes reveal architectural narrowness: 83.9% target the prompt interface (TB1), tool boundaries (TB2b) have zero papers, and two attacker goals, Instruction Hijack and Policy Bypass, account for 87.7% of the corpus.

both are presented as natural language tokens in the same context window [9], [15]. This mixing of instruction and data levels, sometimes referred to as the instruction-data separation problem, means that any text processed by the model can alter its behaviour, regardless of whether that text was intended as an instruction or as data to be processed [8].

The practical consequence is that prompt injection represents a broad class of vulnerabilities rather than a single attack technique. It encompasses direct manipulation of user-side chat interfaces, hostile content embedded in retrieved documents during RAG operations, and malicious payloads delivered via tool outputs or persistent memory in agent-based pipelines [8], [9], [18]. Evaluating this diverse threat landscape requires clear terminological foundations and an understanding of how prompt injection has evolved alongside LLM deployment architectures. The following subsections trace this evolution, define the key terms used in this review, and introduce the standardised frameworks that underpin our analytical approach, in particular Trust Boundaries and MITRE ATLAS.

A. EVOLUTION OF PROMPT INJECTION IN LLMs

The problem of prompt injection has developed alongside the growing capabilities and complexity of LLM deployment, particularly as LLMs have been integrated into applications that retrieve external content and invoke tools or APIs [8], [19], [20]. Early language models such as GPT-2 and BERT primarily functioned as text completion or classification systems with limited interactive capabilities. Security concerns at the time were framed in terms of traditional machine learning threats such as training data poisoning and adversarial-example robustness [9], [21]. The attack surface was comparatively narrow: prompt injection was largely associated with adversaries directly interacting with a model interface, whereas modern LLM-integrated applications additionally ingest untrusted retrieved data (e.g., via RAG) and thus enable indirect prompt injection at inference time [8], [19], [20].

The emergence of instruction-tuned models and conversational interfaces changed this landscape. Following the release of ChatGPT and the subsequent proliferation of instruction-following LLMs, prompt injection and jailbreaking entered both public and academic discourse [8], [22]. Early demonstrations showed that users could bypass safety restrictions by overriding the intended control prompt via natural-language strategies such as role-play and privilege-escalation style prompts (e.g., "sudo mode"), often resembling social-engineering-style instruction overrides [22]. Later threat models distinguish these direct attacks from indirect prompt injection, where malicious instructions arrive via external application inputs (e.g., retrieved documents) rather than the user's own query. This distinction motivates the trust-boundary classifications adopted in this review [8], [19], [20].

As LLM capabilities expanded into tool use, function calling, and agentic workflows, prompt injection risk translated into real-world impact through unintended actions such as unsafe API calls or unauthorised data access, rather than only harmful text generation [8], [20]. In parallel, persistent memory and long-term context mechanisms introduced opportunities for adversarial instructions to persist beyond a single interaction, enabling cross-session influence on model behaviour [8]. Recent literature extends the threat model into specialised settings such as multimodal systems and high-stakes domains, underscoring that prompt injection is not a single attack but an evolving vulnerability class shaped by deployment architecture [21].

Table 1 summarises recent surveys addressing prompt injection and LLM security. Most treat prompt injection as one threat among many or lack systematic methodology. None jointly apply trust-boundary analysis and adversarial-objective alignment across the full corpus, which is the approach adopted here.

B. KEY CONCEPTS AND DEFINITIONS

The prompt injection literature uses inconsistent terminology [9], [22]. We define the core concepts underpinning this review.

1) PROMPT INJECTION

Prompt injection is the insertion of adversarial instructions into an LLM's context window to alter model behaviour in ways unintended by the system operator [8], [19]. This encompasses *direct* injection via the model's input interface and *indirect* injection via external data sources processed during retrieval, tool use, or memory access [20]. Prompt injection is distinguished from jailbreaking, which specifically targets safety-alignment circumvention. Prompt injection is the broader category [9].

2) TRUST BOUNDARIES

A trust boundary is the inference-time interface at which attacker-controlled content first enters the model's decision

TABLE 1. Comparative overview of existing surveys addressing prompt injection and LLM security threats, summarizing each work's scope, number of papers reviewed, and the analytical framework applied to categorize the literature.

Authors	Year	# of papers included	Summary	Structure applied to categorize literature
Kumar et al. [23]	2025	Not specified	Surveys LLM-driven security for multimedia data (image, text, video, audio) in IoT environments. Classifies attacks by training/testing phase, catalogues defence mechanisms, and comparatively discusses LLM architectures for security tasks. Prompt injection is treated as one attack among many without granular sub-classification.	(1) Attack Phase (training vs. testing) (2) Defence Phase (training vs. testing) (3) LLM Architecture Suitability (4) Multimedia Data Type (5) Protection Scheme Type
Ferrag et al. [21]	2025	Not specified	Broad GenAI/LLM-in-cybersecurity survey covering multi-domain applications, empirical benchmarking (42 LLMs on cybersecurity knowledge, 19 on hardware security), OWASP-aligned vulnerability cataloguing including prompt injection, and deployment/optimization techniques.	(1) Cybersecurity application-domain taxonomy (2) Model landscape (general vs. code LLMs; comparative tables) (3) Vulnerability taxonomy aligned to OWASP LLM Top 10 (4) Dataset lifecycle framing (5) Deployment/optimization techniques (quantization, PEFT, RAG)
Aminou et al. [24]	2025	Not specified	Narrative overview of prompt injection in LLM-integrated ATS for resume screening. Contrasts offensive vs. defensive approaches, categorizes defences into detection-based and prevention-based, and identifies three resume-embedded attack patterns (command injection, instruction override, context manipulation).	(1) Attack Orientation: Offensive vs. Defensive (2) Injection Type: Direct vs. Indirect (3) Defence Class: Detection-based vs. Prevention-based (4) ATS Attack Patterns: Command injection; instruction override; context manipulation
Kumar et al. [9]	2024	Not specified	Three-axis framework classifying prompt injection by prompt space (embedding → explored), trust boundary (TB1: alignment / direct attacks; TB2: tools / indirect attacks), and attacker expertise (novice–expert+). Concludes defences remain reactive.	(1) Prompt Injection Spaces: Embedding / Token / Word / Sentence / Explored (2) Trust Boundaries: TB1 (direct) vs. TB2 (indirect) (3) Attack Taxonomy: Direct vs. Indirect with attack families (4) Attacker Proficiency: Novice–Expert+ mapped to spaces (5) Mitigation & Future Directions
Liu et al. [14]	2025	Not specified	Surveys attacks on LVLMs across four co-equal families: adversarial, jailbreak, prompt injection, and data poisoning/backdoors. Prompt injection is subdivided only into unimodal vs. multimodal injection, without addressing trust boundaries, indirect injection, or agent/RAG-specific pathways.	(1) Background & Resources: preliminaries, challenges, tools, datasets, metrics, defences (2) Attack Families: adversarial, jailbreak, prompt injection (unimodal vs. multimodal), data poisoning/backdoors (3) Domain-specific applications: autonomous driving, robotics, medical
Maity & Arora [15]	2024	Not specified	Broad overview of LLM security threats covering prompt injection, DoS, data poisoning, and insecure output handling. Each category is paired with suggested preventive controls. Identifies gaps in open-source model testing and advanced defence techniques.	(1) Attack Classification Dual-Axis: Attack Type vs. Attack Phase (2) Vulnerability–Control Pairing per category (3) Literature Gap → Future Research Direction mapping
Rathod et al. [25]	2025	Not specified	Narrative survey of LLM security and privacy risks covering prompt injection, data poisoning, insecure output handling, and DoS. Discusses mitigations (LLM firewalls, OWASP controls, differential privacy, federated learning) with a healthcare case study using STRIDE threat modeling.	(1) Security Threats: prompt injection, data poisoning, insecure output handling, DoS (2) Privacy Concerns: data leakage, bias, over-reliance (3) Mitigations: LLM firewalls, OWASP practices, DP/FL, HITL (4) Domain Application: healthcare + STRIDE (5) Future Directions: adaptive frameworks, privacy-preserving computation
Samonte et al. [13]	2024	13	Reviews prompt injection vulnerabilities in LLM-integrated chatbots, examining four defence mechanisms (StruQ, AdaShield, Robustness Fine-Tuning, Signed-Prompt) and synthesizing them into a conceptual Event-Driven Architecture. No independent empirical validation is provided.	(1) Vulnerability Assessment of LLM attack surfaces (2) Attack technique examples from cited works (3) Tabular comparison of four named defences (4) Architectural synthesis into an Event-Driven Architecture

pathway. Kumar [9] distinguishes TB1 (direct attacks on the alignment boundary) from TB2 (indirect attacks via external data).

3) ATTACKER GOALS

Attacks are characterised by their ultimate objective. We adopt five goal categories derived from the MITRE ATLAS framework [16]: *Instruction Hijack*, *Policy Bypass*, *Collection/Exfiltration*, *Impact*, and *Persistence*.

III. METHODOLOGY

A. OVERVIEW

This section outlines the methodological steps adopted for this study, which comprises two primary components: (i) a systematic literature review on prompt injection in LLM-integrated systems, and (ii) a three-axis coding framework for synthesising the identified papers. The former describes how relevant studies were identified, screened, and included following PRISMA 2020 guidelines [17]. The latter describes the approach for classifying each included study along three complementary dimensions: trust-boundary entry point, ATLAS-aligned attacker goal, and research orientation.

B. SYSTEMATIC LITERATURE REVIEW PROCEDURE

We conducted a SLR following the Preferred Reporting Items for Systematic Reviews and Meta-analyses (PRISMA) guidelines by Page et al. [17] to achieve this goal. A SLR systematically gathers, assesses, and structures all available and relevant evidence on a specific topic. Our objective was to survey and structure the existing knowledge, classify it

using theoretical frameworks, and identify limitations and research gaps. Besides these tasks, a SLR also aims to provide insights and guidelines for practitioners in this field. Therefore, we defined all existing knowledge as the published literature in specific databases and at a particular time to assure the quality and actuality of our study. We structured our methodology into eight steps derived and adapted from the PRISMA guidelines to ensure a comprehensive SLR. These steps are illustrated in figure 2.

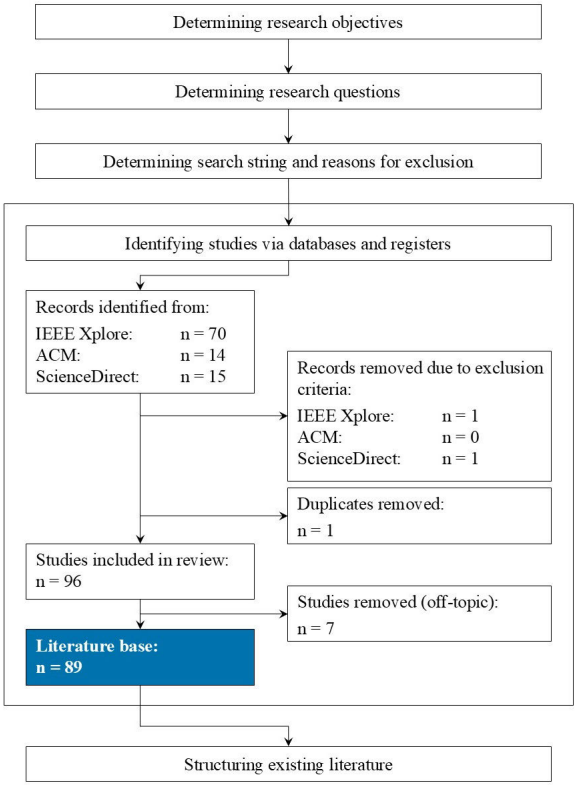


FIGURE 2. Overview of the process for obtaining the literature base as derived from the PRISMA guidelines [17].

C. LITERATURE SEARCH STRATEGY

Following the presented framework, the next step of this SLR was to define the search string and the criteria to exclude individual search results. Our search string consists of the following components: **Title, Abstract & Keywords**: (''LLM*'' OR ''large language model*'') AND (''prompt injection'' OR ''adversarial prompt'')

As this study investigates prompt injection and adversarial prompt attacks targeting LLMs, the search string comprises two components: the target system and the attack vector. The terms ''LLM*'' and ''large language model*'' capture the target system, connected by ''OR'' to account for the abbreviation and its expanded form. The terms ''prompt injection'' and ''adversarial prompt,'' also connected by ''OR,'' capture the two predominant terminologies for input manipulation attacks. The ''AND'' operator ensures only papers addressing both dimensions are retrieved. The wildcard (*) was applied to the system terms to capture plural forms and variations, but not to the attack terms, as these are established compound phrases whose meaning could be diluted by morphological expansion.

To ensure the quality of the identified work, the following exclusion criteria were defined:

Exclusion Criteria

- The publication was published before 2015.
- The publication type is not a peer-reviewed conference paper or journal article. To ensure academic rigor, formats such as poster sessions, editorials, interviews, commentaries, and research-in-progress papers are excluded.
- The publication is considered ''off-topic'' by at least two authors. This criterion ensures the relevance of the included literature.
- The publication is written in a language other than English.

In the fourth step, the databases used for the SLR were selected and searched. The present study used three databases: IEEE Xplore Digital Library, ACM Digital Library, and ScienceDirect database. The search term combination remained the same for all databases and was applied to search titles, abstracts, and keywords. IEEE Xplore was last accessed on 01/10/2025, ACM on 01/10/2025, and ScienceDirect on 01/10/2025.

1) STUDY SELECTION AND EXTRACTION

Before screening, one article from IEEE Xplore and one article from ACM were excluded due to incorrect article types. One additional article was removed because it was a duplicate.

Thus, a total of 96 articles were screened. In the screening process, the authors examined the title and the abstract for the content and semantic relationship to the research question of this paper. To avoid bias in the selection process, all identified papers were individually reviewed by two reviewers.

First, all papers were rechecked for terms contained in the search string. After this first check, the title and abstract were checked using the exclusion criteria controlling the semantic context of the respective work. These papers passing this check were downloaded and stored for information extraction.

After review by the authors, the reliability of the results was 100%. No automation tools were used for the search.

As part of the screening process, several articles were excluded to ensure the relevance and alignment of the selected literature with the objectives of this study. A total of 7 articles were deemed not relevant to this work for the following reasons:

- **Off-topic scope:** Several papers addressed topics only marginally related to LLM security, including gender and ethnic bias, fairness in AI-generated images,

financial technology applications, resource-efficient fine-tuning strategies, and the validation of LLM-based quiz questions. These studies did not address prompt injection or adversarial prompting.
- **No technical attack or defence contribution:** A study examined users' acceptance and performance expectations of generative models without presenting technical attacks or defences against LLMs.

In total, 89 articles were included in the preliminary work and used for the SLR (IEEE Xplore (n = 64), ACM (n = 13), ScienceDirect (n = 12)). Of the 89 included publications, 8 are survey or review papers that inform the comparative landscape in Section II but are excluded from the three-axis coding analysis, yielding an analytical corpus of 81 papers.

D. CLASSIFICATION FRAMEWORK

Each paper in the corpus was coded along three axes: trust boundary (*where* attacker-controlled content first enters the system's decision/action pathway), ATLAS attacker goal (*what* the adversary ultimately seeks to achieve, with a finer record of the operational pattern by which that objective is realised), and research orientation (*whether* the paper primarily contributes an attack, a defence, or a conceptual/framework-level contribution). Cross-tabulating these axes exposes coverage gaps across attack surfaces and attacker objectives, and reveals how the balance between offensive and defensive research is distributed across the corpus.

Several alternative frameworks were considered. A single-axis taxonomy by attack type, as in Kumar et al. [9], cannot expose the cross-tabulation gaps central to RQ1–RQ3. An application-domain taxonomy following Ferrag et al. [21] does not distinguish architectural exposure points within domains. A direct mapping to OWASP LLM01 [26] aligns with practitioner categories but collapses the trust-boundary distinctions this review aims to surface. A pure ATLAS technique-level coding across the 60 retained entries was too granular for 81 papers, leaving most cells empty. The three-axis framework combines architectural exposure, attacker intent at the goal level, and contribution type, with the operational pattern refinement recovering technique-level granularity.

1) AXIS 1: TRUST BOUNDARY CLASSIFICATION

The trust boundary axis classifies each paper by the inference-time interface at which attacker-controlled content first enters the system's decision/action pathway. Kumar et al. [9] propose a two-boundary model distinguishing direct attacks on the alignment boundary (TB1), where system instructions and alignment constraints are expected to govern behaviour at inference time, from indirect attacks that exploit external support systems (TB2), such as code interpreters, browsers, and file readers. Greshake et al. [8] show that indirect prompt injection can propagate via retrieved content, tool/connector execution, and persistent state in LLM-integrated applications.

Building on these foundations, we operationalise the external boundary into a four-category scheme that reflects the architectural surfaces of contemporary LLM-integrated systems.

- **TB1 (Prompt Interface):** Attacker instructions enter via direct user input at inference time, such as chat input, API prompt fields, or user-supplied instructions. This corresponds to the alignment boundary (TB1) of Kumar et al. [9].
- **TB2a (Retrieval Boundary):** Attacker instructions enter via external content retrieved and embedded at inference time, including RAG corpora, webpages, emails, or documents. The distinguishing property is that malicious instructions originate from retrieved context rather than the user prompt.
- **TB2b (Tool/Connector Boundary):** The attacker induces unintended tool, plugin, function, or API calls in tool-augmented or agentic pipelines, including tool-choice hijack, parameter manipulation, or unauthorised tool actions.
- **TB3 (Persistent Memory Boundary):** The attacker plants or exploits stored instructions that persist beyond the current interaction, such as long-term memory, saved profiles, or agent caches, enabling cross-session influence.

Figure 3 illustrates the four trust boundaries and their relationship within an LLM-integrated system architecture.

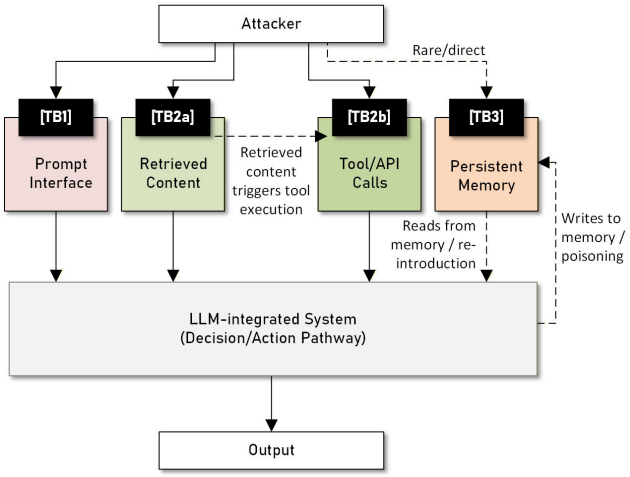


FIGURE 3. Trust boundary classification for LLM-integrated systems. Solid arrows denote primary attacker entry; dashed arrows denote cross-boundary chaining and the TB3 persistence cycle.

Each paper was assigned exactly one primary trust boundary based on the first security-relevant state transition in the system's decision/action pathway. When retrieved content is merely incorporated as context, the primary boundary is TB2a; when retrieved content causes tool invocation or parameterised tool actions, the primary boundary is TB2b because the first such transition is tool execution. When

a paper discussed multiple trust boundaries and ambiguity persisted, a deterministic tie-break priority was applied: TB3 ≻ TB2b ≻ TB2a ≻ TB1, reflecting the principle that boundaries implying stronger architectural capability take precedence over prompt-only inputs: persistence over tool execution, tool execution over retrieval, and retrieval over direct prompting. Papers where the trust boundary was not explicitly stated were flagged as *boundary unclear* and classified based on the best-supported boundary using the tie-break rule.

Papers employing training-time methods that enable or facilitate an inference-time attack, such as fine-tuning to lower jailbreak resistance, were coded based on where the inference-time attack manifests and flagged as *training component present*.

2) AXIS 2: ATLAS ATTACKER GOAL CLASSIFICATION

The second axis classifies each paper by its dominant attacker objective. MITRE ATLAS 5.4.0 [16] defines 97 techniques and 58 sub-techniques, spread across 16 tactics. Of these, we retained 60 (41 techniques and 19 sub-techniques) relating to operations during the inference phase. The remaining 95 deal with reconnaissance, resource development, supply chain compromise, traditional cyber operations, or model access methods that fall outside the scope of prompt injection. The 60 retained entries were grouped into five operational goal categories, which are based on the attacker's ultimate objective rather than on individual techniques:

- **Instruction Hijack:** Adversarial control over the LLM's instruction stream. Direct, indirect, and triggered prompt injection (AML.T0051 and subtechniques).
- **Policy Bypass:** Circumvention of safety filters, alignment guardrails, or content policies, including jailbreaking (AML.T0054) and prompt obfuscation (AML.T0068).
- **Collection/Exfiltration:** Extraction of system prompts, PII, credentials, or RAG contents (AML.T0056, AML.T0057).
- **Impact:** Downstream harm through unauthorised actions, data destruction, or service disruption (AML.T0048 and subtechniques).
- **Persistence:** Adversarial influence maintained across sessions via memory poisoning, context poisoning, or self-replicating prompts (AML.T0080, AML.T0061).

Figure 4 illustrates the decision-making logic separating the attack mechanism from the coded attacker goal.

Each paper was assigned one primary goal and an optional secondary goal distinct from the primary, or none. The primary goal reflects the paper's main contribution; the secondary captures a clearly demonstrated second objective. Framework papers were coded by the dominant attack objective emphasised.

The coding separates the attack *mechanism* from the attacker's *ultimate objective*: injection used to bypass safety constraints is coded as `Policy_Bypass`, injection used to exfiltrate data as `Collection_Exfil`, injection causing downstream harm through unauthorised actions as `Impact`, and injection studied as the phenomenon itself as `Instruction_Hijack`. Defence papers inherit the attacker goal they defend against. Only attacks persisting beyond a single session qualify as `Persistence`.

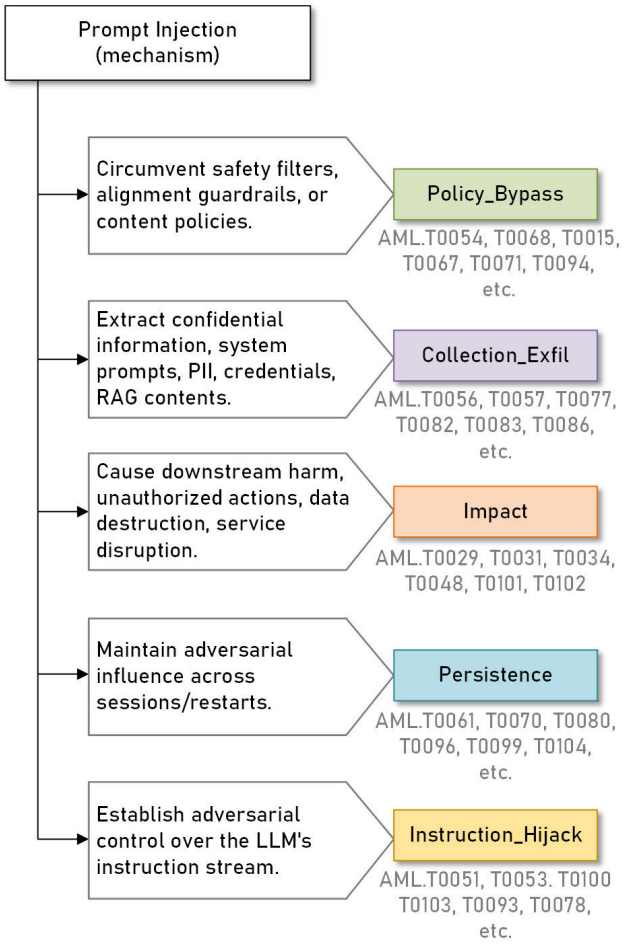


FIGURE 4. ATLAS goal decision-making logic. Prompt injection serves as the common mechanism; the coded goal is determined by the attacker's ultimate objective.

Papers with ambiguous goals were classified based on the dominant objective supported by the text.

a: OPERATIONAL PATTERNS WITHIN GOAL CATEGORIES

The ATLAS goal coding records the attacker's ultimate objective. Within each goal category, the operational pattern by which that objective is realised varies considerably: a Policy_Bypass goal may be pursued through a one-line instruction override or through a gradient-optimised adversarial suffix transferred from a surrogate model. To record this within-goal variation, each paper's ATLAS coding was refined with a primary operational pattern, drawn from the seventeen patterns observed in the corpus, and a secondary pattern where a second pattern is load-bearing. The patterns are listed below in three tiers reflecting empirical frequency and architectural reach.

b: TIER A: DOMINANT PATTERNS

- **M1 Persona Jailbreak (AML.T0054):** Instructions that tell the model to adopt a persona, character, or alter ego whose rules differ from the deployed policy, including DAN-style alter egos and developer-mode framings.
- **M2 Instruction Override (AML.T0054):** A direct natural-language instruction contradicting the system prompt, typically opened by imperatives such as 'ignore' or 'disregard'. No persona, gradient signal, or hidden channel is required.
- **M3 Gradient-Optimised Suffix (AML.T0065 or AML.T0054):** A token sequence derived by gradient-based optimisation and appended to a malicious instruction. Suffixes are obtained via direct gradient access (white-box) or transferred from a surrogate (grey-box).
- **M4 Algorithmic Crafting (AML.T0065):** Adversarial prompts generated by a non-gradient algorithmic procedure such as reinforcement learning, neural architecture search, or attribution-guided synthesis.
- **M5 Indirect Document Injection (AML.T0051.001):** A malicious instruction placed in a document, web page, or email that the application retrieves and incorporates into its context at inference time. This is the canonical TB2a pattern, following Greshake et al. [8].
- **M17 Conceptual or Benchmark (UNMAPPED):** Residual category for papers that do not implement a specific operational technique, covering taxonomies, surveys, conceptual proposals, and benchmark suites that aggregate prior attacks. Orientation is typically Other.

c: TIER B: SECONDARY PATTERNS

- **M6 Hidden-Channel Injection (AML.T0068):** The malicious instruction is concealed in a channel parsed by the model but not readily visible to a human reviewer or surface-level filter, such as CSS-hidden HTML, typographic image rendering, or Unicode encoding.
- **M7 Multi-Turn Manipulation (AML.T0054):** The malicious objective is decomposed across distinct user turns separated by model responses, each turn benign in isolation, so that context drift moves the model into a compliant state. Single-turn nested instructions remain M1 or M2.
- **M8 Rule-Based Perturbation (AML.T0051.000):** Hand-crafted or rule-based character-, word-, or sentence-level perturbations introduced into a benign prompt to alter model output without changing the apparent meaning. Optimisation-derived token sequences are coded M3.
- **M9 In-Context Manipulation (AML.T0051.000):** Corruption of in-context examples or supporting context lists, such as flipped labels in few-shot demonstrations or poisoned entries in contact-name and playlist contexts. The injection point is the example slot rather than the user instruction or a retrieved document.
- **M10 CPS Command Hijack (AML.T0051.000):** A direct command issued through the human-text interface of a cyber-physical system (CPS), typically an LLM-driven robot or an LLM-orchestrated SQL-backed application, whose effect is a state transition in a downstream physical or backend system.
- **M16 Mixed-Pattern Suite (AML.T0051 or AML.T0054):** Heterogeneous evaluation suites, principally automated red-teaming frameworks, that exercise three or more distinct patterns drawn from M1 to M15 without a single pattern carrying the central methodological weight. This coverage must be the paper's own contribution rather than an aggregation of prompts contributed by others.

d: TIER C: ARCHITECTURAL-FRONTIER PATTERNS

- **M11 Tool-Call Injection (AML.T0053):** The attacker steers the model into invoking an unintended tool, function, or API call, or into passing crafted parameters to a legitimate one. Prompt-to-SQL injections forcing unauthorised SQL through a chatbot's database connector are the principal corpus example.
- **M12 MitM Prompt Tampering(AML.T0051.001):** Man-in-the-middle (MitM) network-layer interception and modification of the prompt or response stream between an LLM-integrated client and the model application programming interface (API), typically targeting structured payloads such as JSON request bodies.
- **M13 RAG Poisoning (AML.T0070):** Malicious content planted directly in the retrieval corpus or vector index of a RAG system, so that future queries retrieve and condition on poisoned material. M13 differs from M5 in injection point: M5 places content where retrieval will reach it; M13 modifies the database the retriever queries against.
- **M14 Memory Poisoning (AML.T0080.000):** Adversarial instructions written into the application's long-term memory store and reintroduced into the context window across sessions. A single successful injection compromises subsequent independent interactions until the memory is cleared.
- **M15 Affective Manipulation (AML.T0054):** The malicious request is framed through emotional appeal such as urgency, distress, or moral weight that pressures the model into compliance. Coded as primary only when the affective framing is the paper's methodological contribution; otherwise secondary.

e: TIE-BREAK PRIORITY FOR HYBRID CASES

For papers exhibiting features of more than one pattern, the following priority rules ensure reproducible coding. M3 takes precedence over M6 and M8 when token sequences are optimisation-derived rather than rule-based or hidden-channel. M1 takes precedence over M2 when a persona is invoked. M7 requires distinct conversational turns separated by model responses; nested single-turn instructions remain

M1 or M2. M10 is coded by attack entry point, M11 by attack target, regardless of how the prompt arrived. M16 covers papers contributing pattern breadth themselves; M17 covers papers aggregating or evaluating prompts contributed by others. Trust Boundary and operational pattern are independent dimensions: Pedro et al. [27] is coded TB1 and M11 simultaneously, since the malicious instruction enters via the chatbot prompt but the attack target is the SQL function-call layer.

3) AXIS 3: RESEARCH ORIENTATION CLASSIFICATION

The third axis classifies each paper by its primary research orientation. Every paper in the corpus was assigned to exactly one of three categories:

- **Attack:** The paper primarily demonstrates or evaluates an offensive technique, such as a new prompt injection method or jailbreak strategy.
- **Defence:** The paper primarily proposes or evaluates a protective mechanism, such as an injection detector or guardrail system.
- **Other:** The paper is primarily a taxonomy, conceptual or evaluation framework, risk analysis, or general discussion without contributing a concrete new attack or defence.

When a paper presents both an attack and a defence, the category was determined by the primary contribution. A paper that demonstrates an attack and briefly discusses possible countermeasures was coded as Attack. A paper that evaluates a defence by testing attacks against it was coded as Defence, since the attack serves only as the evaluation condition. Benchmarking papers that systematically test attacks without proposing a new defence were coded as Attack. Framework and guidance papers without a concrete offensive or defensive contribution were coded as Other.

IV. RESULTS

A. CORPUS OVERVIEW

The final literature base comprises 81 peer-reviewed publications drawn from three databases: IEEE Xplore, ACM Digital Library, and ScienceDirect. Tables 2 and 3 present the complete literature base.

Fig. 5 shows the distribution over time. Research output grew from 4 publications in 2023 to 33 in 2024 and 40 in 2025, with 4 early-access publications dated 2026. The sharp increase between 2023 and 2024 coincides with the widespread deployment of LLM-integrated applications and the corresponding emergence of prompt injection as a recognised attack vector.

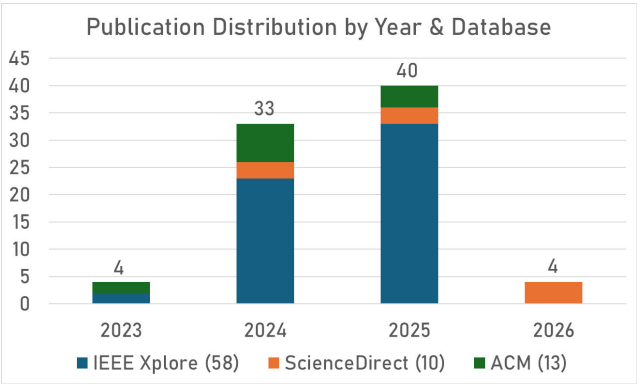


FIGURE 5. Number of publications in the literature base structured by year and Database.

B. TRUST BOUNDARY DISTRIBUTION (AXIS 1)

The classification of the trust boundary shows a pronounced concentration at the prompt interface: TB1 accounts for 68 of 81 papers (83.9%), TB2a for 12 (14.8%), and TB3 for a single paper (1.2%). No paper in the corpus addresses the tool/connector boundary. This complete absence is notable given the widespread use of tool-augmented LLM systems in production environments.

C. ATTACKER GOAL DISTRIBUTION (AXIS 2)

Instruction Hijack dominates as primary goal with 47 papers (58.0%), followed by Policy Bypass with 24 (29.6%), Impact with 5 (6.2%), Collection/Exfil with 4 (4.9%), and Persistence with 1 (1.2%).

Secondary goal analysis shows a similar pattern: Instruction Hijack appears as the secondary goal in 27 papers (33.3%), followed by Policy Bypass (14; 17.3%), Impact (14; 17.3%), Collection/Exfil (6; 7.4%) and Persistence (2; 2.5%). Eighteen papers (22.2%) were assigned no secondary goal. The frequent assignment of Instruction Hijack as a secondary goal reflects its role as an enabling mechanism for most attacker objectives, which is consistent with the distinction between mechanism and objective specified in the coding rules.

D. CROSS-TABULATION ANALYSIS

1) TRUST BOUNDARY × PRIMARY GOAL

Table 4 presents a cross-tabulation of trust boundaries against primary attacker goals. A single cell - TB1 × Instruction Hijack - comprises 38 articles (46.9% of the corpus). Together with TB1 × Policy Bypass (23 articles), research on prompt interfaces covers 75.3% of all publications. TB2a is distributed across Instruction Hijack (9), Collection/Exfil (1), Impact (1), and Policy Bypass (1), indicating greater goal diversity at the retrieval boundary despite a smaller sample size (see Figure 6). The only TB3 publication targets Persistence. All five goal columns are empty for TB2b.

2) TRUST BOUNDARY × RESEARCH ORIENTATION

Table 5 cross-tabulates trust boundaries against research orientation. At TB1, defence-oriented papers (41 of 68) outnumber attack-oriented contributions (20 of 68), with 7 papers classified as Other. At TB2a, attack and defence contributions are distributed almost evenly (6 attack, 5 defence, 1 other), reflecting the relatively recent emergence of

TABLE 2. Overview of the publications in the literature base. The table presents the authors and publication year of each study, along with trust boundary (Axis 1), primary/secondary ATLAS attacker goal (Axis 2) and the research orientation (Axis 3).

Author(s)	Year	Trust Boundary	Primary Attacker Goal	Secondary Attacker Goal	Orientation
Abdelkader et al. [28]	2025	TB1	Policy Bypass	-	Defence
Abdelnabi et al. [29]	2025	TB2a	Instruction Hijack	-	Defence
Alla et al. [30]	2025	TB1	Instruction Hijack	Impact	Defence
AlOmar et al. [31]	2025	TB1	Instruction Hijack	Policy Bypass	Other
Ayachitula et al. [32]	2024	TB1	Instruction Hijack	Persistence	Other
Balas et al. [33]	2024	TB1	Policy Bypass	Impact	Attack
Bao et al. [34]	2025	TB1	Policy Bypass	Instruction Hijack	Defence
Brach et al. [35]	2025	TB2a	Collection/Exfil	Instruction Hijack	Defence
CeldIr et al. [36]	2025	TB1	Instruction Hijack	Impact	Attack
Chernyshev et al. [37]	2023	TB1	Instruction Hijack	Impact	Defence
Chernyshev et al. [38]	2024	TB2a	Instruction Hijack	-	Defence
Chisty et al. [39]	2024	TB1	Instruction Hijack	-	Defence
Dinu et al. [40]	2025	TB2a	Instruction Hijack	Impact	Attack
Donato et al. [41]	2025	TB1	Policy Bypass	Instruction Hijack	Other
Erguen et al. [42]	2025	TB1	Instruction Hijack	Impact	Defence
Erguen et al. [43]	2025	TB1	Instruction Hijack	Policy Bypass	Defence
Filho et al. [44]	2024	TB1	Instruction Hijack	-	Other
Fu et al. [45]	2025	TB1	Instruction Hijack	Policy Bypass	Attack
Goh et al. [46]	2025	TB1	Policy Bypass	Instruction Hijack	Defence
Greshake et al. [8]	2023	TB2a	Instruction Hijack	Impact	Attack
Hao et al. [47]	2025	TB1	Instruction Hijack	Policy Bypass	Defence
Herrador et al. [18]	2026	TB3	Persistence	Collection/Exfil	Attack
Jacob et al. [20]	2024	TB1	Instruction Hijack	-	Defence
Jadhav et al. [48]	2025	TB1	Instruction Hijack	-	Defence
Jang et al. [49]	2025	TB1	Instruction Hijack	Policy Bypass	Defence
Johnson et al. [50]	2024	TB1	Instruction Hijack	Policy Bypass	Defence
Joseph et al. [51]	2025	TB1	Instruction Hijack	Collection/Exfil	Defence
Joshi et al. [52]	2025	TB1	Instruction Hijack	Impact	Defence
Kim et al. [53]	2024	TB1	Instruction Hijack	Collection/Exfil	Defence
Kulkarni et al. [54]	2025	TB1	Policy Bypass	Instruction Hijack	Defence
Kwon et al. [55]	2024	TB1	Policy Bypass	Instruction Hijack	Attack
Labunets et al. [56]	2025	TB2a	Instruction Hijack	Policy Bypass	Attack
Lan et al. [57]	2025	TB1	Instruction Hijack	Policy Bypass	Defence
Li et al. [58]	2024	TB1	Impact	Instruction Hijack	Attack
Liu et al. [22]	2024	TB1	Policy Bypass	Instruction Hijack	Attack
Liu et al. [59]	2025	TB1	Instruction Hijack	-	Defence
Marinelli et al. [60]	2025	TB1	Instruction Hijack	Policy Bypass	Defence
Marulli et al. [61]	2024	TB1	Instruction Hijack	Collection/Exfil	Other
Mittal et al. [62]	2025	TB1	Policy Bypass	Instruction Hijack	Defence
Mo et al. [63]	2025	TB2a	Impact	Persistence	Attack
Muliarevych et al. [64]	2024	TB1	Instruction Hijack	Collection/Exfil	Defence
Muliarevych et al. [65]	2024	TB1	Instruction Hijack	Collection/Exfil	Defence
Nair et al. [66]	2024	TB1	Policy Bypass	Instruction Hijack	Defence
Omri et al. [67]	2024	TB1	Policy Bypass	Instruction Hijack	Defence
Ong et al. [68]	2024	TB1	Impact	-	Attack
Pasquini et al. [19]	2024	TB2a	Instruction Hijack	-	Attack
Pedro et al. [27]	2025	TB1	Instruction Hijack	Impact	Attack
Pokhrel et al. [69]	2024	TB1	Instruction Hijack	Policy Bypass	Defence

TABLE 3. Overview of the publications in the literature base (Part II).

Author(s)	Year	Trust Boundary	Primary Attacker Goal	Secondary Attacker Goal	Orientation
Rahman et al. [70]	2024	TB1	Instruction Hijack	-	Defence
Rahman et al. [71]	2025	TB1	Instruction Hijack	-	Defence
Raj et al. [72]	2025	TB1	Instruction Hijack	Policy Bypass	Defence
Rosari et al. [73]	2024	TB1	Instruction Hijack	Policy Bypass	Defence
Russinovich et al. [74]	2025	TB2a	Policy Bypass	Instruction Hijack	Other
Shaikh et al. [75]	2025	TB1	Impact	Instruction Hijack	Attack
Shan et al. [76]	2024	TB1	Policy Bypass	Instruction Hijack	Attack
Sharma et al. [77]	2024	TB1	Instruction Hijack	Policy Bypass	Defence
Shen et al. [78]	2024	TB1	Policy Bypass	Instruction Hijack	Attack
Shi et al. [79]	2024	TB2a	Instruction Hijack	Impact	Attack
Shi et al. [80]	2025	TB1	Instruction Hijack	-	Defence
Shi et al. [81]	2026	TB1	Policy Bypass	Instruction Hijack	Attack
Shibli et al. [82]	2024	TB1	Policy Bypass	Impact	Attack
Shu et al. [83]	2025	TB1	Policy Bypass	-	Attack
Song et al. [84]	2023	TB1	Instruction Hijack	Impact	Defence
Tien et al. [85]	2024	TB1	Policy Bypass	Instruction Hijack	Defence
Traykov et al. [86]	2024	TB1	Policy Bypass	Instruction Hijack	Defence
Vajpayee et al. [87]	2024	TB1	Collection/Exfil	Instruction Hijack	Other
Wadgaonkar et al. [88]	2025	TB1	Policy Bypass	Instruction Hijack	Attack
Wang et al. [89]	2024	TB1	Collection/Exfil	Instruction Hijack	Attack
Wang et al. [90]	2025	TB1	Instruction Hijack	Policy Bypass	Defence
Wang et al. [91]	2026	TB1	Instruction Hijack	-	Defence
Yang et al. [6]	2025	TB1	Policy Bypass	Instruction Hijack	Attack
Yang et al. [92]	2025	TB1	Policy Bypass	Instruction Hijack	Defence
Yang et al. [93]	2025	TB1	Collection/Exfil	Instruction Hijack	Defence
Yi et al. [94]	2025	TB2a	Instruction Hijack	-	Defence
Yi et al. [95]	2025	TB1	Policy Bypass	Instruction Hijack	Defence
Ying et al. [96]	2025	TB1	Policy Bypass	Instruction Hijack	Attack
Yip et al. [97]	2023	TB1	Instruction Hijack	Impact	Attack
Zhang et al. [98]	2024	TB1	Instruction Hijack	Impact	Defence
Zhang et al. [99]	2024	TB1	Instruction Hijack	-	Attack
Zhang et al. [100]	2026	TB1	Impact	Instruction Hijack	Defence
Zhu et al. [101]	2024	TB1	Instruction Hijack	-	Other

TABLE 4. Trust boundary × primary attacker goal. IH = instruction hijack, PB = policy bypass, IM = impact, CE = Collection/Exfil, PE = persistence.

	IH	PB	IM	CE	PE	Total
TB1	38	23	4	3	0	68
TB2a	9	1	1	1	0	12
TB2b	0	0	0	0	0	0
TB3	0	0	0	0	1	1
Total	47	24	5	4	1	81

TABLE 5. Trust boundary × research orientation.

	Attack	Defence	Other	Total
TB1	20	41	7	68
TB2a	6	5	1	12
TB2b	0	0	0	0
TB3	1	0	0	1
Total	27	46	8	81

retrieval-boundary research where attack characterisation and defence development proceed in parallel. The single TB3 paper is classified as Attack, consistent with the exploratory nature of research on an emerging attack surface.

3) PRIMARY GOAL × RESEARCH ORIENTATION

Table 6 cross-tabulates primary attacker goals against research orientation. Within Instruction Hijack, the largest goal category, defence papers (32 of 47) outnumber attack

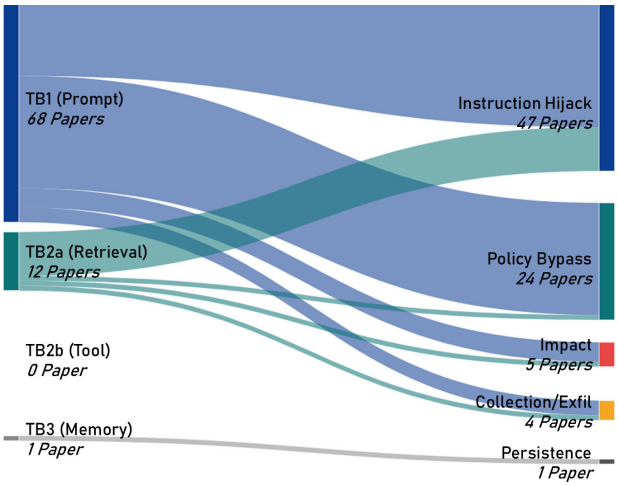


FIGURE 6. Flow diagram of research distribution across trust boundaries (left) and primary attacker goals (right). Flow width is proportional to the number of papers. The absence of flows from TB2b and the near-invisible TB3 thread visualise the architectural coverage gap central to RQ1 and RQ2. Exact counts are reported in Table 4.

papers (10 of 47), indicating that the research community has moved towards building countermeasures for the most studied injection type. Policy Bypass shows a balanced split (11 attack vs. 11 defence). In contrast, Impact is predominantly attack-oriented (4 of 5), which is consistent with the fact that demonstrating downstream harm requires constructing offensive scenarios. Collection/Exfil and Persistence, with 4 and 1 papers respectively, are too small for meaningful orientation analysis.

TABLE 6. Primary attacker goal × research orientation.

	Attack	Defence	Other	Total
Instruction Hijack	10	32	5	47
Policy Bypass	11	11	2	24
Impact	4	1	0	5
Collection/Exfil	1	2	1	4
Persistence	1	0	0	1
Total	27	46	8	81

4) RESEARCH ORIENTATION × YEAR

Figure 7 shows the distribution of research orientation over time. In 2023, the corpus contained 4 papers with an even split between attack and defence (2 each). By 2024, the corpus grew to 33 papers, with attack contributions at 12 (36.4%) and defence at 16 (48.5%). In 2025, the shift becomes more pronounced: of 40 papers, 26 (65.0%) are defence-oriented while 11 (27.5%) focus on attacks. The 2026 early-access papers (n=4) are too few for trend interpretation. Excluding 2026, the data suggests that as the field matures, research effort is redirecting from attack demonstration towards the development of protective mechanisms.

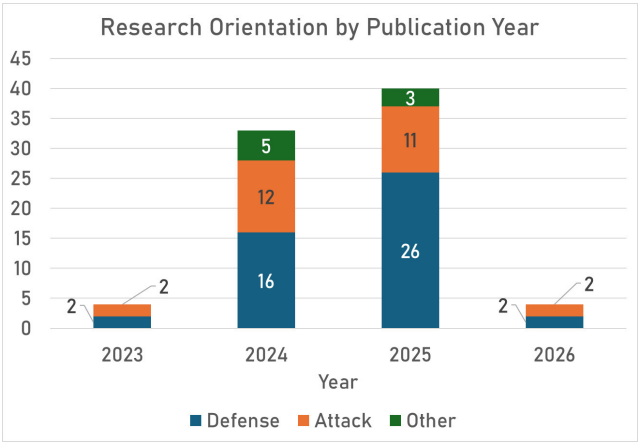


FIGURE 7. Distribution of research orientation (Attack, Defence, Other) by publication year. The increasing share of defence-oriented publications reflects a maturing research field.

5) OPERATIONAL PATTERN DISTRIBUTION

Within Axis 2, each paper was assigned a primary operational pattern from the seventeen patterns defined in Section III-D2. Table 7 reports the primary distribution across the corpus, ordered by tier and by descending count within each tier. Where a paper exhibits a load-bearing second pattern, the secondary assignment is recorded in the appendix but excluded from the count to keep the totals interpretable as a proportion of the corpus. The complete per-paper assignment is provided in Appendix (Tables 8-12).

The distribution concentrates heavily in Tier A and thins out across Tiers B and C. Two patterns within Tier A account for almost two fifths of the corpus on their own, with three further patterns covering gradient-based, non-gradient algorithmic, and retrieval-based techniques making up most of the remainder. Tier B contributes operational variants such as obfuscation, multi-turn drift, and rule-based perturbation, none reaching the frequency of the dominant Tier A patterns. Tier C is uniformly sparse, with every architectural-frontier pattern represented by a single paper.

Three observations follow. First, the largest single category is M17, recording papers that contribute a taxonomy, benchmark, or framework rather than a specific operational technique. The frequency of conceptual articles in itself provides insight into publishing practices within this field. Second, the Tier A concentration parallels the TB1 concentration on Axis 1: empirical research clusters around a small set of well-understood operational patterns on both axes. Third, the Tier C uniformity at single-paper coverage mirrors the TB2b absence and TB3 sparsity findings reported in Section IV-B. Architectural-frontier patterns receive sparse research attention regardless of which axis is used to view them, suggesting a shared infrastructure and reproducibility constraint rather than an axis-specific barrier.

The M10 category warrants a separate note. CPS Command Hijack covers three papers, all addressing LLM-driven robotics or LLM-orchestrated SQL-backed applications. This

TABLE 7. Primary operational pattern distribution across the corpus, grouped by tier and ordered by descending count within tier.

Tier	ID	Pattern	n (%)
A	M17	Conceptual or Benchmark	17 (21.0)
A	M2	Instruction Override	15 (18.5)
A	M1	Persona Jailbreak	8 (9.9)
A	M5	Indirect Document Injection	7 (8.6)
A	M4	Algorithmic Crafting	6 (7.4)
A	M3	Gradient-Optimised Suffix	5 (6.2)
B	M6	Hidden-Channel Injection	5 (6.2)
B	M7	Multi-Turn Manipulation	4 (4.9)
B	M10	CPS Command Hijack	3 (3.7)
B	M16	Mixed-Pattern Suite	3 (3.7)
B	M8	Rule-Based Perturbation	2 (2.5)
B	M9	In-Context Manipulation	1 (1.2)
C	M11	Tool-Call Injection	1 (1.2)
C	M12	MitM Prompt Tampering	1 (1.2)
C	M13	RAG Poisoning	1 (1.2)
C	M14	Memory Poisoning	1 (1.2)
C	M15	Affective Manipulation	1 (1.2)

small but coherent sub-cluster indicates that cyber-physical and backend-action contexts are an emerging concentration even where the entry point remains the prompt interface.

V. DISCUSSION

The concentration at TB1 and on two attacker objectives is consistent with the incentive structure of security research. Instruction Hijack and Policy Bypass produce immediately verifiable results: a model generates prohibited content or follows an injected instruction in place of its system prompt. These outcomes are reproducible with API access or a local model instance and require no supporting infrastructure beyond the model itself. By contrast, studying exfiltration via retrieved content (TB2a) requires a realistic retrieval pipeline with sensitive data [8]. Studying tool-boundary attacks (TB2b) requires a functioning agent with permission boundaries and callable tools. Evaluating Persistence (TB3) requires multi-session infrastructure capable of measuring cross-session state propagation [18]. The experimental effort scales with architectural complexity, but the publication incentive does not scale proportionally. The corpus therefore largely reflects what researchers are able to reproduce using the available infrastructure, rather than what poses the greatest risk in production systems. This asymmetry is self-reinforcing: the absence of TB2b and TB3 research within the peer-reviewed corpus retrieved under our SLR protocol means there are no shared benchmarks, no baseline results to improve on, and no established evaluation protocols, all of which further increase the barrier to entry for researchers targeting these boundaries.

The defence-dominant pattern at TB1 (41 of 68 papers) could be read as evidence that the prompt interface is approaching adequate coverage. This reading does not hold under closer examination. Defence dominance at TB1 is consistent with a maturing research area where attack characterisation preceded defence development, as indicated by the temporal shift from balanced attack-defence output in 2023 to 65.0% defence orientation in 2025 (Figure 7). It does not indicate that existing defences are effective or sufficient. The maturity pattern is entirely absent at other boundaries. At TB2a, where attack and defence contributions are nearly balanced (6 vs. 5), research is still in an early characterisation phase where attack demonstration and defence proposal proceed in parallel. TB2b and TB3 have not entered this phase within the corpus retrieved under our SLR protocol. The field therefore exhibits a maturity gradient across trust boundaries: TB1 has moved into defence development, TB2a remains in exploratory characterisation, and TB2b and TB3 have not started. This gradient distorts the aggregate picture: the overall corpus is 56.8% defence-oriented, but 41 of those 46 defence papers target TB1 (Table 5). Organisations operating tool-augmented or memory-enabled systems cannot derive security guidance from this body of work, because the boundaries relevant to their architectures are either absent from the corpus (TB2b) or represented by a single paper (TB3).

The architectural narrowness identified across all three axes points to an underlying paradigm constraint. The research community treats prompt injection as a property of the model rather than as a property of the system in which the model operates. Studies at TB1 test whether a model produces unintended outputs in response to adversarial inputs. The model functions as both the attack surface and the evaluation target. In production deployments, the model is one component in an orchestration layer that includes retrieval pipelines, tool registries, permission systems, and persistent state stores [9]. Attacks at TB2b and TB3 do not target the model's language capabilities; they exploit the trust relationships between the model and its surrounding infrastructure. A tool-choice hijack succeeds not because the model fails to detect an injection but because the orchestration layer grants the model unconstrained tool access. A memory poisoning attack persists not because the model lacks safety alignment but because the persistence layer does not validate the origin of stored instructions [18]. Practitioner frameworks have already adopted this systems-level view: the OWASP Top 10 for LLM Applications [26] treats prompt injection (LLM01) as a system-level vulnerability spanning tools and external content, and separately classifies excessive tool permissions and autonomy (LLM06: Excessive Agency) as a distinct risk category. The academic corpus retrieved has not yet reached this level of architectural differentiation. Closing this gap requires threat models that account for inter-component trust, test environments that simulate realistic orchestration architectures, and evaluation criteria that

TABLE 8. Per-paper operational pattern assignment (Part I of V): primary pattern, optional secondary pattern, parent MITRE ATLAS technique identifier, and short attack description. Dashes in the ATLAS column indicate conceptual or framework papers with no parent technique; dashes in the secondary-pattern column indicate that no load-bearing second pattern was identified.

Author(s)	ATLAS ID	Primary Pattern	Secondary Pattern	Short Attack Description
Abdelkader et al. [28]	AML.T0054	M2	-	Harmful prompts and off-topic queries submitted directly to the language model to bypass content moderation and elicit prohibited responses.
Abdelnabi et al. [29]	AML.T0051.001	M5	-	Natural language instructions embedded in external data sources manipulate the language model to deviate from the primary user task and execute unauthorised commands.
Alla et al. [30]	AML.T0051.000	M2	-	Deceptive instructions embedded directly into user prompts to bypass system safeguards, extract sensitive data, and manipulate model behaviour.
AlOmar et al. [31]	UNMAPPED	M17	-	Curriculum framework integrating generative AI into university cybersecurity education, covering defensive applications and offensive techniques like prompt injection and data poisoning.
Ayachitula et al. [32]	UNMAPPED	M17	-	Label Noise Injection Attacks manipulate few-shot training examples by flipping class labels within the prompt to degrade classification accuracy.
Balas et al. [33]	AML.T0054	M15	-	Adversarial prompts presenting innocuous requests submitted via direct chat to bypass safety alignment and elicit harmful instructions for ocular warfare.
Bao et al. [34]	AML.T0054	M2	M3	Adversarial jailbreak prompts, including gradient-optimised and template-based variants, submitted directly to bypass value alignment and elicit harmful responses.
Brach et al. [35]	AML.T0068	M6	M5	Defensive prompt injection embeds hidden text instructions within HTML elements using CSS display properties to disrupt large language models performing automated web scraping.
CeldIr et al. [36]	AML.T0054	M1	M15	Hateful Goal Hijacking injects negative emotional directives into moral dilemma prompts to manipulate ethical decision-making in reasoning models.
Chernyshev et al. [37]	AML.T0051.000	M10	M11	Prompt-to-SQL (P2SQL) attacks embed malicious instructions into natural language queries submitted to an LLM-integrated application to execute unauthorized backend database operations.
Chernyshev et al. [38]	AML.T0051.001	M5	-	Malicious instructions embedded within external data sources to hijack autonomous agent functionality and bypass intended operational constraints.
Chisty et al. [39]	AML.T0051.000	M2	-	Malicious instructions submitted via direct user prompts to manipulate language model behaviour, leak sensitive data, or compromise the system.
Dinu et al. [40]	AML.T0068	M6	M5	Hidden HTML instructions embedded within web pages retrieved by scraping agents to force the language model to terminate response generation and output a specific keyword.
Donato et al. [41]	UNMAPPED	M17	-	Framework for systematically evaluating the adversarial robustness of large language models against text-based prompt hacking and jailbreak attacks.
Erguen et al. [42]	AML.T0051.000	M8	-	Adversarial prompt manipulations, including prefix, insertion, and suffix perturbations, injected into user queries to induce factually incorrect, misleading, or logically contradictory responses from language models.
Erguen et al. [43]	AML.T0051.000	M8	-	Adversarial prompts containing misleading word-level or sentence-level insertions submitted via direct input to manipulate context and elicit biased responses.

measure harm at the system level rather than at the model output level.

A. KEY CHALLENGES AND OPEN ISSUES

Three challenges emerge. First, a benchmark deficit for TB2b and TB3: without standardised tool schemas, authorisation models, and deterministic simulators, research on these boundaries will remain limited. Second, evaluation metrics for outcome-oriented objectives are poorly specified. Attack success rate alone does not capture the severity of an exfiltration event, the side effects of an unauthorised action, or the duration of a persistence window. Third, there is a persistent conflation of mechanism and objective: an injection is performed and referred to as the objective,

TABLE 9. Per-paper operational pattern assignment (Part II of V), continued from Table 8.

Author(s)	ATLAS ID	Primary Pattern	Secondary Pattern	Short Attack Description
Filho et al. [44]	UNMAPPED	M17	-	Framework for risk management of generative AI systems assessing vulnerabilities like prompt injection across multiple language models using automated scanners.
Fu et al. [45]	AML.T0065	M4	-	HijackNet generates adaptive adversarial prompts via neural architecture search and reinforcement learning, optimising token perturbations to bypass model defences and hijack target tasks.
Goh et al. [46]	AML.T0051	M16	-	Automated red-teaming framework generating adversarial prompts spanning roleplay, indirect document injection, multi-turn manipulation, and obfuscation for risk evaluation.
Greshake et al. [8]	AML.T0051.001	M5	-	Indirect Prompt Injection embeds malicious instructions into external data sources like websites or emails, which are retrieved and executed by LLM-integrated applications to manipulate behaviour.
Hao et al. [47]	AML.T0054	M2	-	Direct user prompts containing malicious injection templates and prohibited questions submitted to bypass safety alignment and coerce cloud-deployed models into generating illegal content.
Herrador et al. [18]	AML.T0080.000	M14	M5	SpAIware poisons persistent memory via indirect injection in untrusted documents, causing continuous data exfiltration across future sessions through hidden image rendering.
Jacob et al. [20]	AML.T0051.001	M5	M2	Malicious instructions embedded within third-party data sources like emails or web pages subvert the original application prompt to hijack the execution of LLM-integrated applications.
Jadhav et al. [48]	AML.T0054	M2	-	Deceptive instructions injected into language model prompts to bypass intended constraints and manipulate behaviour for unauthorised information disclosure.
Jang et al. [49]	UNMAPPED	M17	-	Framework for proactive security testing of generative AI applications using automated red and blue teaming agents to identify and mitigate prompt injection vulnerabilities.
Johnson et al. [50]	AML.T0054	M2	-	Injects malicious instructions via an emulated shell interface to bypass honeypot constraints, reveal the underlying AI, or execute arbitrary host commands.
Joseph et al. [51]	UNMAPPED	M17	-	Framework for evaluating open-source large language model security using custom probes, automated vulnerability scanners, and fine-tuned models to detect prompt injections and code vulnerabilities.
Joshi et al. [52]	AML.T0054	M2	-	Direct prompt injections containing override keywords such as ignore or discard submitted to financial LLM applications to bypass authorization and execute fraudulent transactions.
Kim et al. [53]	AML.T0054	M1	-	Direct prompt injection and jailbreak commands submitted to bypass safety filters, forcing the model to generate malware or leak sensitive information.
Kulkarni et al. [54]	AML.T0054	M7	-	Multi-turn conversational prompts gradually shift semantic context to mask malicious intent and bypass single-turn safety filters.
Kwon et al. [55]	AML.T0068	M6	-	Graph-based prompt injection replaces sensitive words with mathematical expressions that draw text on a two-dimensional plane to bypass safety alignment.
Labunets et al. [56]	AML.T0065	M3	M5	Fun-tuning extracts loss signals from remote fine-tuning APIs to guide the discrete optimisation of adversarial suffixes, boosting indirect prompt injections to hijack closed-weight models.
Lan et al. [57]	AML.T0054	M2	-	Direct user prompts containing explicit instructions to ignore, override, or bypass prior safety alignments are submitted to elicit harmful content.
Li et al. [58]	AML.T0065	M4	-	AdvPro generates semantic-preserving adversarial code prompts via gradient attribution to induce code completion models to output vulnerable code.
Liu et al. [22]	UNMAPPED	M17	-	Taxonomy and empirical benchmark of 78 jailbreak prompts categorised into 10 patterns and evaluated against ChatGPT to quantify robustness across prohibited scenarios.

even if the demonstrated harm is exfiltration or impact. A reporting standard that specifies input boundaries, attacker objectives, threat model, and success criteria would improve comparability across studies.

TABLE 10. Per-paper operational pattern assignment (Part III of V), continued from Table 9.

Author(s)	ATLAS ID	Primary Pattern	Secondary Pattern	Short Attack Description
Liu et al. [59]	AML.T0051.001	M5	M3	Malicious instructions embedded into untrusted external data sources, such as webpages, to hijack LLM-integrated applications and evade known-answer detection mechanisms via gradient-optimised separators.
Marinelli et al. [60]	AML.T0054	M2	-	Direct user prompts containing malicious instructions or conditional statements submitted to circumvent system pre-prompts and elicit harmful responses or unauthorised code execution.
Marulli et al. [61]	UNMAPPED	M17	-	Survey of the multifaceted role of LLMs in cybersecurity, detailing their applications as defensive guardians, attack enablers, and vulnerable targets.
Mittal et al. [62]	AML.T0051.001	M5	-	Adversarial instructions embedded within Infrastructure-as-Code configuration files to bypass LLM-based security scanners and force the automated approval of vulnerable deployments.
Mo et al. [63]	AML.T0070	M13	-	Broken Bags injects LLM-generated toxic text into RAG databases to mislead retrievers and force incorrect model responses to specific queries.
Muliarevych et al. [64]	AML.T0051.000	M7	M2	Malicious prompts categorised into probing and prompt-chain attacks submitted via direct user interfaces to bypass system constraints and extract sensitive model configurations or memorised data.
Muliarevych et al. [65]	AML.T0054	M7	M2	Targeted probing queries and iterative prompt chains submitted via direct user interfaces to bypass security constraints and extract sensitive model information.
Nair et al. [66]	AML.T0054	M1	-	Role-play instructions and negative sentiment commands submitted via direct user prompts to bypass safety alignment and elicit prohibited or unethical content from the model.
Omri et al. [67]	AML.T0054	M1	M2	Adversarial strings and jailbreak templates appended to malicious user requests via direct prompt interfaces to bypass model alignment and elicit objectionable content.
Ong et al. [68]	UNMAPPED	M17	-	Benchmark framework evaluating social biases and stigmas in large language models using curated adversarial prompt templates to quantify biased responses.
Pasquini et al. [19]	AML.T0065	M3	M5	Neural Exec generates gradient-optimised, inline execution triggers embedded within external documents to bypass retrieval-augmented generation preprocessing and force language models to execute hidden payloads.
Pedro et al. [27]	AML.T0053	M11	M5	P2SQL injections embed malicious natural language instructions into direct chatbot prompts or database records to force LLM-integrated frameworks to generate and execute unauthorized SQL queries.
Pokhrel et al. [69]	AML.T0054	M2	-	Crafts synthetic malicious prompts submitted via direct user input to bypass safety constraints, inject fake reviews, redirect traffic, or secretly collect personal information.
Rahman et al. [70]	AML.T0051	M16	-	Malicious instructions embedded into direct user queries or external web content to hijack language model responses and override predefined system prompts.
Rahman et al. [71]	AML.T0051	M16	-	Malicious instructions embedded within user inputs or external data sources to manipulate the operational output of language models.
Raj et al. [72]	AML.T0054	M2	-	Malicious queries containing prompt injections or adversarial perturbations submitted via direct user interfaces to bypass safety filters, generate prohibited content, and execute unauthorised commands.
Rosari et al. [73]	AML.T0054	M2	M1	Malicious prompt injection payloads submitted via direct user input to bypass system instructions and manipulate the behaviour of a cybersecurity consultation chatbot.
Russinovich et al. [74]	UNMAPPED	M17	-	Survey of three intrinsic risks in large language models, focusing on hallucinations, indirect prompt injections, and jailbreaks, alongside general mitigation strategies.

B. THEORETICAL IMPLICATIONS

The three-axis framework enables gap analysis beyond single-axis taxonomies. Existing surveys classify by trust boundary [9] or application domain [21], but none cross-tabulate architectural exposure with attacker objectives and research orientation. The cross-tabulation surfaces what

TABLE 11. Per-paper operational pattern assignment (Part IV of V), continued from Table 10.

Author(s)	ATLAS ID	Primary Pattern	Secondary Pattern	Short Attack Description
Shaikh et al. [75]	AML.T0051.001	M12	-	Man-in-the-middle network interception alters JSON-formatted prompts and responses between a vacuum robot and a cloud-based LLM to bypass collision avoidance and manipulate motor controls.
Shan et al. [76]	AML.T0054	M3	-	Gradient-optimised adversarial suffix appended to IoT-related user prompts to bypass safety alignment and elicit harmful instructions for exploiting smart devices.
Sharma et al. [77]	AML.T0068	M6	-	Malicious text instructions embedded within images submitted to multi-modal large language models to bypass system specifications and alter chatbot behaviour.
Shen et al. [78]	UNMAPPED	M17	-	Benchmark framework evaluating 1,405 in-the-wild jailbreak prompts collected from online communities against six commercial large language models to quantify their vulnerabilities and safeguard efficacy.
Shi et al. [79]	AML.T0051.001	M3	M5	JudgeDeceiver generates a gradient-optimised adversarial sequence injected into candidate responses to manipulate LLM-as-a-Judge systems into selecting the attacker-controlled text as the optimal answer.
Shi et al. [80]	AML.T0054	M2	-	Malicious instructions embedded within user prompts to disrupt the reasoning process of large language models and induce erroneous or unintended outputs.
Shi et al. [81]	AML.T0054	M6	M1	Virtual Scenario Hypnosis embeds malicious queries into fictional narratives, converting them into optimised typographic images alongside Unicode-encrypted in-context examples to jailbreak vision-language models.
Shibli et al. [82]	AML.T0054	M1	-	AbuseGPT uses persona-based jailbreak prompts submitted via direct chat to bypass ethical guardrails and generate sophisticated SMS phishing campaigns.
Shu et al. [83]	UNMAPPED	M17	-	Framework for evaluating the effectiveness of jailbreak prompts across multiple large language models using coarse-grained and fine-grained scoring metrics.
Song et al. [84]	AML.T0051.000	M9	-	Malicious instructions embedded within user-provided context lists, such as contact names or playlists, to hijack the contextual spelling correction model and force arbitrary transcript outputs.
Tien et al. [85]	AML.T0054	M1	-	Role-play instructions and persona adoption prompts submitted via direct user input to bypass safety filters and elicit prohibited content from educational AI chatbots.
Traykov et al. [86]	AML.T0054	M1	-	Character role-play instructions submitted via direct user prompts to bypass safety alignment and elicit prohibited step-by-step guidance on malware creation for financial gain.
Vajpayee et al. [87]	UNMAPPED	M17	-	Evaluation of commercial and open-source language models regarding the disclosure of sensitive security posture information, proposing a risk quantification framework.
Wadgaonkar et al. [88]	AML.T0065	M4	-	RLAF generates multilingual, code-switched adversarial prompts via reinforcement learning to elicit toxic responses and bypass conventional moderation in target LLMs.
Wang et al. [89]	AML.T0065	M4	-	AUTORED generates synthetic malicious prompts via reinforcement learning to iteratively bypass sandwich defences and extract sensitive access codes from LLMs.
Wang et al. [90]	AML.T0054	M2	-	Adaptive jailbreak payloads containing guessed delimiter escape sequences are submitted via direct user input to break structural constraints and override the agent system instructions.
Wang et al. [91]	UNMAPPED	M17	-	Framework for instruction data mining evaluating language model robustness against character-level, word-level, and sentence-level adversarial prompt perturbations to select high-quality fine-tuning data.
Yang et al. [6]	UNMAPPED	M17	-	Framework for evaluating the security of Chinese large language models across value alignment, robustness, data security, and prompt injection using a two-phase progressive attack strategy.
Yang et al. [92]	AML.T0054	M3	-	AdvPrompter generates human-readable adversarial suffixes appended to malicious instructions to bypass safety alignment and elicit prohibited harmful content.

TABLE 12. Per-paper operational pattern assignment (Part V of V), continued from Table 11.

Author(s)	ATLAS ID	Primary Pattern	Secondary Pattern	Short Attack Description
Yang et al. [93]	AML.T0054	M1	-	Adversarial prompt injection queries submitted via simulated shell interfaces to probe the system's true nature and expose the underlying LLM-driven honeypot environment.
Yi et al. [94]	AML.T0051.001	M5	-	Malicious instructions embedded within retrieved external content manipulate LLM-integrated applications to produce irrelevant responses or execute targeted commands.
Yi et al. [95]	AML.T0054	M2	-	Adversarial jailbreak templates and toxic prompts submitted via direct user input to bypass safety alignments and elicit harmful, insulting, or profane language from the model.
Ying et al. [96]	AML.T0065	M4	M6	BAP constructs bi-modal adversarial prompts by embedding universal perturbations into images and iteratively optimizing textual queries via Chain-of-Thought reasoning to jailbreak aligned vision-language models.
Yip et al. [97]	UNMAPPED	M17	-	Evaluation framework for assessing large language model resilience against prompt injection attacks via a curated dataset and automated impact scoring.
Zhang et al. [98]	AML.T0051.000	M10	-	Malicious navigational instructions injected via the human text interface of LLM-controlled mobile robots to hijack operational goals and cause physical collisions or task failures.
Zhang et al. [99]	AML.T0065	M4	-	G2PIA generates semantically similar adversarial sentences under KL-divergence constraints, inserted into user prompts to force incorrect model answers.
Zhang et al. [100]	AML.T0051.000	M10	-	Obvious Malicious Injection and Goal Hijacking Injection manipulate human instruction interfaces in LLM-driven robots to force hazardous physical movements and bypass navigation safety constraints.
Zhu et al. [101]	UNMAPPED	M17	-	Framework for evaluating large language models, integrating prompt engineering, dynamic evaluation protocols, and multiple levels of adversarial prompt attacks.

no individual axis reveals: TB2b is entirely absent, and the defence-oriented pattern at TB1 does not hold at TB2a, where research is still evenly split between attack and defence.

The results support a conceptual shift: prompt injection is transitioning from a linguistic attack on individual models to a system-level attack on orchestration layers spanning retrieval, tool execution, and persistent state. Future frameworks should treat it as a systems-security problem requiring trust boundary analysis and state provenance, not solely as a prompt engineering challenge.

The operational pattern distribution refines the goal-level coding. The five ATLAS categories record what the attacker seeks and the seventeen patterns record how that objective is realised. Tier A concentration shows that empirical research clusters on a small set of well-understood techniques, predominantly direct instruction overrides, persona-based jailbreaks, and gradient-optimised or algorithmically crafted suffixes. The same papers driving TB1 dominance on Axis 1 drive Tier A dominance here, reflecting a shared focus on prompt-level techniques that require no architectural infrastructure to reproduce. Tier C uniformity at single-paper coverage extends the architectural-narrowness finding into the operational dimension: the infrastructure missing for TB2b and TB3 research is the same infrastructure that would support studying tool-call injection, memory poisoning, RAG poisoning, and man-in-the-middle tampering at scale. The prevalence of M17 papers indicates that a substantial portion of the field's output is conceptual rather than empirical.

Trust Boundary and operational pattern are independent dimensions and may differ from one another in terms of their architectural positioning. Pedro et al. [27] is the principal corpus example, coded TB1 and M11 simultaneously because the malicious instruction enters through the chatbot prompt but the attack target is the SQL function-call layer. The operational pattern axis nonetheless exhibits the same frontier sparsity as the Trust Boundary axis: M14 (Memory Poisoning) is exercised by a single paper, Herrador et al. [18], the same paper that anchors TB3. This $n = 1$ status mirrors the TB3 result and supports the interpretation that architectural frontiers receive sparse research attention regardless of the analytical axis used.

C. PRACTICAL IMPLICATIONS

First, organisations deploying tool-augmented or memory-enabled LLM systems operate on surfaces for which there are no peer-reviewed studies in our corpus. Securing these boundaries must rely on internal security measures such as red team testing, sandbox testing and audit logging, supplemented by guidance from vendors, until specific research findings are available. Second, security controls should be context-specific: input filtering and output validation for TB1, retrieval sanitisation and content provenance for TB2a.

For TB3 and TB2b, where the corpus contains one and zero papers respectively, the identification of appropriate controls remains an open research question. Third, it should not be assumed that detection benchmarks validated for hijacking at the prompt level can be generalised to targets such as exfiltration or persistence. Fourth, the concentration of defence research at TB1 (41 of 46 defence papers) should not be taken as evidence of coverage at other boundaries. Organisations operating at TB2b or TB3 cannot rely on published defences within the scope of this review, as these are either absent from the corpus (TB2b) or represented by a single paper (TB3).

VI. CONCLUSION

This work presented a systematic literature review of 81 peer-reviewed publications on prompt injection in LLM-integrated systems, analysed through a three-axis coding framework combining trust boundary classification, MITRE ATLAS attacker goal mapping with an operational pattern refinement, and research orientation. The review yields four principal findings. First, 83.9% of research targets the prompt interface (TB1), while tool-boundary attacks (TB2b) are entirely absent from the corpus and persistent memory attacks (TB3) are represented by a single paper. Second, two attacker objectives, Instruction Hijack and Policy Bypass, account for 87.7% of primary goals, leaving data exfiltration, downstream impact, and persistent compromise collectively marginalised. Third, the corpus is defence-oriented (56.8%), but this defence concentration is localised at TB1 and Instruction Hijack. Defence work grew from 50% of publications in 2023 to 65.0% in 2025, while attack surfaces beyond the prompt interface (TB2b, TB3) and objectives such as Persistence remain underexplored, with TB2b absent from the corpus entirely and Persistence represented by a single paper. Fourth, the operational pattern refinement of Axis 2 reveals the same architectural-frontier sparsity within the technique dimension, with each Tier C pattern represented by a single paper.

These findings indicate that the field is architecturally narrow, with research concentrated on attack surfaces and objectives that do not reflect the multi-component architecture of production LLM systems. The three-axis framework provides a reusable coding scheme for tracking how this distribution changes as tool-augmented and memory-enabled agents enter wider deployment.

A. LIMITATIONS

Five limitations should be considered when interpreting these results. First, the search strategy targets ''prompt injection'' and ''adversarial prompt'' terminology; relevant work framed under alternative terms (e.g., ''tool hijacking,'' ''agent security,'' ''memory poisoning'') may not have been retrieved. Consequently, the absence of TB2b and TB3 papers should be interpreted as a terminological gap rather than an empirical one. Research on these boundaries may exist under labels such as tool hijacking or agent security that fall outside the search string. This is particularly likely given that the field's vocabulary has not yet stabilised around the architectural distinctions that define TB2b and TB3. Second, the review is limited to three digital libraries (IEEE Xplore, ACM Digital Library, ScienceDirect); additional venues may host relevant work not indexed in these sources. Third, each paper was assigned a single primary trust boundary based on the first security-relevant state transition, which may compress multi-boundary attack chains into a single label. Fourth, single-coder classification, while guided by deterministic coding rules and quality flags, does not eliminate subjectivity. Fifth, the corpus of 81 papers reflects the state of the literature at the time of database access (October 2025); given the rapid pace of publication in this domain, the distributions reported here may shift as new work appears.

B. FUTURE RESEARCH

The gaps identified in this review define a concrete research agenda. First, dedicated empirical studies of tool-boundary (TB2b) attacks are needed, supported by standardised testbeds that specify tool schemas, permission models, and deterministic execution environments; although tool abuse was demonstrated as early as 2023, no subsequent study isolates TB2b as a primary entry boundary [8]. Second, attacks on persistent memory (TB3) require long-term evaluation protocols that measure persistence duration, reinfection rate, and transmission between sessions, building on the only existing precedent in this area [18]. Third, goal-aligned evaluation metrics should be developed for outcome-centric objectives, including exfiltration severity scoring for Collection/Exfiltration, side-effect quantification for Impact, and persistence window measurement for Persistence. Fourth, the Tier C operational patterns identified in this review (M11 to M15) warrant dedicated empirical work, since each currently rests on a single corpus paper despite covering production-relevant surfaces such as tool-call injection, memory poisoning, and RAG poisoning. Fifth, extending the coding framework to include secondary trust boundaries alongside secondary goals, beyond the two-boundary model proposed by Kumar et al. [9], would enable a more detailed analysis of attack chains across multiple boundaries. Finally, a standardised reporting checklist for prompt injection research, specifying model versions, threat models, artefacts, boundary-related configurations and goal-aligned success metrics, would improve comparability between studies [102].

APPENDIX

PER-PAPER OPERATIONAL PATTERN ASSIGNMENT

This appendix reports the complete per-paper assignment of the operational pattern classification introduced in Section III-D2a. Each row lists the primary pattern, the secondary pattern where a second pattern is load-bearing, the parent MITRE ATLAS technique identifier, and a short attack description extracted during the coding process.

Trust-boundary and research-orientation assignments for each paper are reported in Tables 2 and 3.

REFERENCES

[1] K. K. Singhal et al., "Toward expert-level medical question answering with large language models," *Nature Med.*, vol. 31, no. 3, pp. 943–950, Jan. 2025.

[2] B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, and A. Fawzi, "Mathematical discoveries from program search with large language models," *Nature*, vol. 625, no. 7995, pp. 468–475, Dec. 2023.

[3] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. E. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, "Training language models to follow instructions with human feedback," in *Proc. 36th Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 27730–27744.

[4] A. J. Thirunavukarasu, D. S. J. Ting, K. Elangovan, L. Gutiérrez, T. F. Tan, and D. S. W. Ting, "Large language models in medicine," *Nature Med.*, vol. 29, no. 8, pp. 1930–1940, Jul. 2023.

[5] F. Xing, K. Du, G. Mengaldo, E. Cambria, and R. Welsch, "AI reshaping financial modeling," *npj Artif. Intell.*, vol. 1, no. 1, pp. pp. 1–6, Oct. 2025.

[6] Y. Yang, Z. Huang, Z. Jia, and J. Fang, "A comprehensive security evaluation framework for Chinese large language models," in *Proc. ACM Web Conf.*, May 2025, pp. 2426–2433.

[7] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Conf. Secur. Symp.*, 2023, pp. 1831–1847.

[8] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what You've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop Artif. Intell. Secur.*, Nov. 2023, pp. 79–90.

[9] S. S. Kumar, M. L. Cummings, and A. Stimpson, "Strengthening LLM trust boundaries: A survey of prompt injection attacks surender suresh kumar Dr. M.L. cummings Dr. Alexander stimpson," in *Proc. IEEE 4th Int. Conf. Human-Machine Syst. (ICHMS)*, May 2024, pp. 1–6.

[10] J. Clusmann, D. Ferber, I. C. Wiest, C. V. Schneider, T. J. Brinker, S. Foersch, D. Truhn, and J. N. Kather, "Prompt injection attacks on vision language models in oncology," *Nature Commun.*, vol. 16, no. 1, pp. 1–9, Feb. 2025.

[11] Y. Yang, Q. Jin, F. Huang, and Z. Lu, "Adversarial prompt and fine-tuning attacks threaten medical large language models," *Nature Commun.*, vol. 16, no. 1, pp. 1–10, Oct. 2025.

[12] A. I. Alzahrani, "PromptGuard a structured framework for injection resilient language models," *Sci. Rep.*, vol. 16, no. 1, p. 1277, Jan. 2026.

[13] M. J. C. Samonte, J. E. R. Aparize, E. J. S. Gonzales, and J. L. Morilla, "Data breach prevention in AI systems: Employing event-driven architecture to combat prompt injection attacks in chatbots," in *Proc. IEEE 12th Int. Conf. Inf.*, Aug. 2024, pp. 626–632.

[14] D. Liu, M. Yang, X. Qu, P. Zhou, Y. Cheng, and W. Hu, "A survey of attacks on large vision–language models: Resources, advances, and future trends," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 11, pp. 19525–19545, Nov. 2025.

[15] S. Maity and J. Arora, "The colossal defense: Security challenges of large language models," in *Proc. IEEE Delhi Sect. Flagship Conf. (DELCON)*, Nov. 2024, pp. 1–5.

[16] MITRE Corporation. (2026). *ATLAS: Adversarial Threat Landscape for AI Systems*. [Online]. Available: https://atlas.mitre.org/

[17] M. J. Page et al., "The PRISMA 2020 statement: An updated guideline for reporting systematic reviews," *BMJ*, vol. 372, p. 71, Sep. 2021.

[18] M. Herrador and J. Rehberger, "SpAIware: Uncovering a novel artificial intelligence attack vector through persistent memory in LLM applications and agents," *Future Gener. Comput. Syst.*, vol. 174, Jan. 2026, Art. no. 107994.

[19] D. Pasquini, M. Strohmeier, and C. Troncoso, "Neural exec: Learning (and learning from) execution triggers for prompt injection attacks," in *Proc. Workshop Artif. Intell. Secur.*, Nov. 2024, pp. 89–100.

[20] D. Jacob, H. Alzahrani, Z. Hu, B. Alomair, and D. Wagner, "PromptShield: Deployable detection for prompt injection attacks," in *Proc. 15th ACM Conf. Data Appl. Secur. Privacy*, Jun. 2025, pp. 341–352.

[21] M. A. Ferrag, F. Alwahedi, A. Battah, B. Cherif, A. Mechri, N. Tihanyi, T. Bisztray, and M. Debbah, "Generative AI in cybersecurity: A comprehensive review of LLM applications and vulnerabilities," *Internet Things Cyber-Physical Syst.*, vol. 5, pp. 1–46, Jan. 2025.

[22] Y. Liu, G. Deng, Z. Xu, Y. Li, Y. Zheng, Y. Zhang, L. Zhao, T. Zhang, and K. Wang, "A Hitchhiker's guide to jailbreaking ChatGPT via prompt engineering," in *Proc. 4th Int. Workshop Softw. Eng. AI Data Quality Cyber-Physical Systems/Internet Things*, Jul. 2024, pp. 12–21.

[23] A. Kumar, M. M. Salim, D. Camacho, and J. H. Park, "A comprehensive survey on large language models for multimedia data security: Challenges and solutions," *Comput. Netw.*, vol. 267, pp. 1–24, Jul. 2025.

[24] L. Aminou, A. Daaif, M. Soulami, and M. Youssfi, "'Ignore all and accept my resume': The impact of prompt injection in automatic resume screening," in *Proc. 5th Int. Conf. Innov. Res. Appl. Sci., Eng. Technol. (IRASET)*, May 2025, pp. 1–5.

[25] V. Rathod, S. Nabavirazavi, S. Zad, and S. S. Iyengar, "Privacy and security challenges in large language models," in *Proc. IEEE 15th Annu. Comput. Commun. Workshop Conf. (CCWC)*, Jan. 2025, pp. 00746–00752.

[26] OWASP Foundation. (2024). *OWASP Top 10 for Large Language Model Applications*. [Online]. Available: https://genai.owasp.org/

[27] R. Pedro, M. E. Coimbra, D. Castro, P. Carreira, and N. Santos, "Prompt-to-SQL injections in LLM-integrated web applications: Risks and defenses," in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng. (ICSE)*, Apr. 2025, pp. 1768–1780.

[28] H. Abdelkader, M. Abdelrazek, S. Singh, I. Logothetis, P. Rani, R. Vasa, and J.-G. Schneider, "Safeguarding LLM-applications: Specify or train?" in *Proc. IEEE/ACM 4th Int. Conf. AI Eng.-Softw. Eng. AI (CAIN)*, Apr. 2025, pp. 275–276.

[29] S. Abdelnabi, A. Fay, G. Cherubin, A. Salem, M. Fritz, and A. Paverd, "Get my drift? Catching LLM task drift with activation deltas," in *Proc. IEEE Conf. Secure Trustworthy Mach. Learn. (SaTML)*, Apr. 2025, pp. 43–67.

[30] S. Alla and A. S. Sichani, "Cyberattacks on large language models–attack detection and architecture adaptability," in *Proc. SoutheastCon*, Mar. 2025, pp. 143–148.

[31] B. AlOmar and Z. Trabelsi, "Integrating generative AI in cybersecurity curricula," in *Proc. IEEE Global Eng. Educ. Conf. (EDUCON)*, Apr. 2025, pp. 1–9.

[32] S. Ayachitula, C. Kundu, and B. Mishra, "Probing robustness of in-context learning in LLM classification predictions under label noise," in *Proc. IEEE 6th Int. Conf. Trust, Privacy Secur. Intell. Syst., Appl. (TPS-ISA)*, Oct. 2024, pp. 427–436.

[33] M. Balas, D. T. Wong, and S. A. Arshinoff, "Artificial intelligence, adversarial attacks, and ocular warfare," *AJO Int.*, vol. 1, no. 3, Oct. 2024, Art. no. 100062.

[34] Z. Bao, Y. Ji, W. Wu, X. Chen, and L. He, "Supervisor alignment framework: Enhancing LLM alignment with query-ignoring strategy and multi-agent interaction," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Apr. 2025, pp. 1–5.

[35] W. Brach, M. Petrik, K. Košt'ál, and M. Ries, "Ghosts in the markup: Techniques to fight large language model-powered web scrapers," in *Proc. 37th Conf. Open Innov. Assoc. (FRUCT)*, May 2025, pp. 37–46.

[36] O. M. Çeldır and G. Dalkiliç, "Sensitivity to emotional exploitation in reasoning models: Stereotypical analysis," in *Proc. 7th Int. Congr. Hum.-Comput. Interact., Optim. Robotic Appl. (ICHORA)*, May 2025, pp. 1–8.

[37] M. Chernyshev, Z. Baig, and R. R. M. Doss, "Towards large language model (LLM) forensics using LLM-based invocation log analysis," in *Proc. 1st ACM Workshop Large AI Syst. Models with Privacy Saf. Anal.*, Nov. 2023, pp. 89–96.

[38] M. Chernyshev, Z. Baig, and R. Doss, "[Short paper] forensic analysis of indirect prompt injection attacks on LLM agents," in *Proc. IEEE 6th Int. Conf. Trust, Privacy Secur. Intell. Syst., Appl. (TPS-ISA)*, Oct. 2024, pp. 409–411.

[39] M. M. O. Chisty, A. N. M. Sakib, M. S. Khan, and M. B. Shuvo, "Prompt injection detection using ensemble of BERT and LSTM with GloVe embeddings," in *Proc. 13th Int. Conf. Electr. Comput. Eng. (ICECE)*, Dec. 2024, pp. 453–458.

[40] I.-V. Dinu, G.-M. Danciu, R. C. Vintilă, and T.-C. Bălan, "Disrupting large language models with hidden prompt injection attacks embedded in HTML pages," in *Proc. Int. Aegean Conf. Electr. Mach. Power Electron. (ACEMP) Int. Conf. Optim. Electr. Electron. Equip. (OPTIM)*, May 2025, pp. 1–10.

[41] J. Donato, "Benchmarking LLM robustness against prompt-based adversarial attacks," in *Proc. 20th Eur. Dependable Comput. Conf. Companion (EDCC-C)*, Apr. 2025, pp. 60–63.

[42] A. E. Ergün and A. Onan, "A2CEM: A contrastive embedding framework for adversarial robustness in large language models," *Knowl.-Based Syst.*, vol. 326, pp. 1–20, Sep. 2025.

[43] A. E. Ergün and A. Onan, "Adversarial prompt detection in large language models: A classification-driven approach," *Comput., Mater. Continua*, vol. 83, no. 3, pp. 4855–4877, 2025.

[44] R. H. Filho and D. Colares, "A methodology for risk management of generative AI based systems," in *Proc. Int. Conf. Softw., Telecommun. Comput. Netw. (SoftCOM)*, Sep. 2024, pp. 1–6.

[45] L. Fu, K. Shih, J. Cao, S. Wu, Y. Jin, and Z. Yang, "Adversarial prompt optimization in LLMs: HijackNet's approach to robustness and defense evasion," in *Proc. 4th Int. Symp. Comput. Appl. Inf. Technol. (ISCAIT)*, 2025, pp. 2081–2085.

[46] A. Goh, B. Chee, M. Vagnoli, L. Baldassarre, and A. Narayan, "Scaling responsible generative AI: Automating red teaming of LLM applications," in *Proc. IEEE Conf. Artif. Intell. (CAI)*, May 2025, pp. 902–905.

[47] G. Hao and J. Wu, "Privacy-preserving prompt injection detection for smart cloud-deployed large language models," in *Proc. IEEE 10th Int. Conf. Smart Cloud (SmartCloud)*, May 2025, pp. 26–31.

[48] G. Jadhav, A. K. Singh, Z. Khanam, and R. Hercock, "A novel GNN-based approach for detection of prompt injection attacks," in *Proc. IEEE Int. Conf. Cyber Secur. Resilience (CSR)*, Aug. 2025, pp. 77–82.

[49] S. K. Jang Bahadur, G. Dhar, and L. Nigam, "GenAI security: Outsmarting the bots with a proactive testing framework," in *Proc. IEEE Conf. Artif. Intell. (CAI)*, May 2025, pp. 610–612.

[50] S. Johnson, R. Hassing, J. Pijpker, and R. Loves, "A modular generative honeypot shell," in *Proc. IEEE Int. Conf. Cyber Secur. Resilience (CSR)*, Sep. 2024, pp. 387–394.

[51] J. K. Joseph, E. Daniel, V. Kathiresan, and M. A. P. Manimegalai, "Prompt injection in large language model exploitation: A security perspective," in *Proc. Int. Conf. Electron., Comput., Commun. Control Technol. (ICECCC)*, May 2025, pp. 1–8.

[52] T. Joshi, V. Naik, I. Mistry, and R. Mangrulkar, "Prevention of prompt injection attacks over financial applications integrated with LLM," in *Proc. 3rd Int. Conf. Advancement Comput. Comput. Technol. (InCACCT)*, Apr. 2025, pp. 225–230.

[53] M. Kim, T. Kwon, K. Shim, and B. Kim, "Protection of LLM environment using prompt security," in *Proc. 15th Int. Conf. Inf. Commun. Technol. Converg. (ICTC)*, Oct. 2024, pp. 1715–1719.

[54] P. Kulkarni and A. Namer, "Temporal context awareness: A defense framework against multi-turn manipulation attacks on large language models," in *Proc. IEEE Conf. Artif. Intell. (CAI)*, May 2025, pp. 930–935.

[55] H. Kwon, J. Kim, and W. Pak, "Graph-based prompt injection attacks against large language models," in *Proc. IEEE Int. Conf. Technol., Informat., Manage., Eng. Environ. (TIME-E)*, Aug. 2024, pp. 1–5.

[56] A. Labunets, N. V. Pandya, A. Hooda, X. Fu, and E. Fernandes, "Fun-tuning: Characterizing the vulnerability of proprietary LLMs to optimization-based prompt injection attacks via the fine-tuning interface," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2025, pp. 411–429.

[57] Q. Lan, A. Kaul, S. Jones, S. Westrum, V. Pandurangan, N. K. Das Pattanaik, and P. Pattanayak, "Hybrid constitutional classifiers for prompt injection defense," in *Proc. IEEE Int. Conf. Electro Inf. Technol. (eIT)*, May 2025, pp. 225–229.

[58] X. Li, G. Meng, S. Liu, L. Xiang, K. Sun, K. Chen, X. Luo, and Y. Liu, "Attribution-guided adversarial code prompt generation for code completion models," in *Proc. 39th IEEE/ACM Int. Conf. Automated Softw. Eng.*, Oct. 2024, pp. 1460–1471.

[59] Y. Liu, Y. Jia, J. Jia, D. Song, and N. Z. Gong, "DataSentinel: A game-theoretic detection of prompt injection attacks," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2025, pp. 2190–2208.

[60] R. Marinelli, "Adversarial prompt filtering based on societal risk," in *Proc. IEEE 25th Int. Symp. Cluster, Cloud Internet Comput. Workshops (CCGridW)*, Aug. 2025, pp. 225–228.

[61] F. Marulli, P. Paganini, and F. Lancellotti, "The three sides of the moon LLMs in cybersecurity: Guardians, enablers and targets," *Proc. Comput. Sci.*, vol. 246, pp. 5340–5348, Nov. 2024.

[62] A. Mittal and V. Venkatesan, "Practical integration of large language models into enterprise CI/CD pipelines for security policy validation: An industry-focused evaluation," in *Proc. IEEE Int. Conf. Service-Oriented Syst. Eng. (SOSE)*, Jul. 2025, pp. 197–203.

[63] Y. Mo, M. Tang, R. Lin, B. Zhou, and X. Li, "Broken bags: Disrupting service through the contamination of large language models with misinformation," *IEEE Access*, vol. 13, pp. 109607–109623, 2025.

[64] O. Muliarevych, "Enhancing system security: LLM-driven defense against prompt injection vulnerabilities," in *Proc. IEEE 17th Int. Conf. Adv. Trends Radioelectronics, Telecommun. Comput. Eng. (TCSET)*, Oct. 2024, pp. 420–423.

[65] O. Muliarevych, "Mitigating input prompt attack vulnerabilities in systems with a language model interface," in *Proc. 14th Int. Conf. Dependable Syst., Services Technol. (DESSERT)*, Oct. 2024, pp. 1–8.

[66] A. Nair, J. Vithayathil, S. P. Nambiar, S. Snehasree, and S. Murali, "Exploring the essence of prompt tone: A comparative study of large language model security," in *Proc. 3rd Int. Conf. Electr., Electron., Inf. Commun. Technol. (ICEEICT)*, Jul. 2024, pp. 1–8.

[67] S. Omri, M. Abdelkader, and M. Hamdi, "MalPID: Malicious prompt injection detection dataset for large language model based applications," in *Proc. IEEE 11th Int. Conf. Commun. Netw. (ComNet)*, Mali, Nov. 2024, pp. 1–5.

[68] Y. J. Ong, J. Gala, S. An, R. Moore, and D. Jadav, "Adversarially exploring vulnerabilities in LLMs to evaluate social biases," in *Proc. IEEE Int. Conf. Big Data (BigData)*, Dec. 2024, pp. 5289–5297.

[69] K. Pokhrel, C. Sanín, M. K. H. Sakib, M. R. Islam, and E. Szczerbicki, "An adversarial machine learning approach on securing large language model with vigil, an open-source initiative," *Proc. Comput. Sci.*, vol. 246, pp. 686–695, Nov. 2024.

[70] M. A. Rahman, H. Shahriar, F. Wu, and A. Cuzzocrea, "Applying pre-trained multilingual BERT in embeddings for improved malicious prompt injection attacks detection," in *Proc. 2nd Int. Conf. Artif. Intell., Blockchain, Internet Things (AIBThings)*, Mali, Sep. 2024, pp. 1–7.

[71] M. A. Rahman, H. Shahriar, G. Francia, F. Wu, A. Cuzzocrea, M. Rahman, M. J. H. Faruk, and S. I. Ahamed, "Fine-tuned large language models (LLMs): Improved prompt injection attacks detection," in *Proc. IEEE 49th Annu. Comput., Softw., Appl. Conf. (COMPSAC)*, Jul. 2025, pp. 1033–1039.

[72] G. Raj, Hamzah, N. Raj, and N. Ranjan, "Hacking LLMs: A technical analysis of security vulnerabilities and defense mechanisms," in *Proc. 2nd Int. Conf. Comput. Intell., Commun. Technol. Netw. (CICTN)*, Feb. 2025, pp. 555–560.

[73] H. S. Rosari, Girinoto, R. Novita Yasa, N. Qomariasih, and R. Anindya Wijayanti, "SiJesi: Large language model chatbot with augmented retrieval approach generation and prompt engineering," in *Proc. Int. Conf. Comput. Eng., Netw., Intell. Multimedia (CENIM)*, Nov. 2024, pp. 1–6.

[74] M. Russinovich, A. Salem, S. Zanella-Béguelin, and Y. Zunger, "The price of intelligence," *Commun. ACM*, vol. 68, no. 9, pp. 46–53, Aug. 2025.

[75] A. Shaikh, A. Varol, and J. Virkki, "From prompts to motors: Man-in-the-middle attacks on LLM-enabled vacuum robots," *IEEE Access*, vol. 13, pp. 137505–137513, 2025.

[76] W. Shan, T. Long, and Z. Zhou, "Adversarial attacks on IoT systems leveraging large language models," in *Proc. Int. Conf. Identificat., Inf. Knowl. Internet Things (IIKI)*, Dec. 2024, pp. 154–159.

[77] R. K. Sharma, V. Gupta, and D. Grossman, "Defending language models against image-based prompt attacks via user-provided specifications," in *Proc. IEEE Secur. Privacy Workshops (SPW)*, May 2024, pp. 112–131.

[78] X. Shen, Z. Chen, M. Backes, Y. Shen, and Y. Zhang, "'Do anything now': Characterizing and evaluating in-the-wild jailbreak prompts on large language models," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Dec. 2024, pp. 1671–1685.

[79] J. Shi, Z. Yuan, Y. Liu, Y. Huang, P. Zhou, L. Sun, and N. Z. Gong, "Optimization-based prompt injection attack to LLM-as-a-judge," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Dec. 2024, pp. 660–674.

[80] L. Shi, Y. Kang, J. Hu, X. Li, and M. Yang, "Meticulous thought defender: Fine-grained chain-of-thought (CoT) for detecting prompt injection attacks of large language models," *IEEE Access*, vol. 13, pp. 113194–113207, 2025.

[81] X. Shi, S. Chen, G. Zhang, W. Wei, Y. Li, Z. Fan, and J. Liu, "Jailbreak attack with multimodal virtual scenario hypnosis for vision-language models," *Pattern Recognit.*, vol. 172, pp. 1–16, Apr. 2026.

[82] A. M. Shibli, M. M. A. Pritom, and M. Gupta, "AbuseGPT: Abuse of generative AI ChatBots to create smishing campaigns," in *Proc. 12th Int. Symp. Digit. Forensics Secur. (ISDFS)*, Apr. 2024, pp. 1–6.
[83] D. Shu, C. Zhang, M. Jin, Z. Zhou, and L. Li, "AttackEval: How to evaluate the effectiveness of jailbreak attacking on large language models," *ACM SIGKDD Explorations Newslett.*, vol. 27, no. 1, pp. 10–19, Jul. 2025.
[84] G. Song, Z. Wu, G. Pundak, A. Chandorkar, K. Joshi, X. Velez, D. Caseiro, B. Haynor, W. Wang, N. Siddhartha, P. Rondon, and K. C. Sim, "Contextual spelling correction with large language models," in *Proc. IEEE Autom. Speech Recognit. Understand. Workshop (ASRU)*, Dec. 2023, pp. 1–8.
[85] L. A. Tien and P. Van Huong, "Optimizing transformer models for prompt jailbreak attack detection in AI assistant systems," in *Proc. 1st Int. Conf. Cryptography Inf. Secur. (VCRIS)*, Dec. 2024, pp. 1–4.
[86] K. Traykov, "A framework for security testing of large language models," in *Proc. IEEE 12th Int. Conf. Intell. Syst. (IS)*, Aug. 2024, pp. 1–7.
[87] P. Vajpayee, C. Karuppiah, and G. Hossain, "Chat GPT and cyber risks: Need of custom large language models," in *Proc. IEEE 3rd Int. Conf. Data, Decis. Syst. (ICDDS)*, Dec. 2024, pp. 1–4.
[88] V. S. Wadgaonkar and N. Ratha, "An adversarial toxicity prompt generator exploiting multilingual code-switching in LLMs," in *Proc. IEEE Conf. Artif. Intell. (CAI)*, May 2025, pp. 884–887.
[89] Z. Wang and M. A. Tayebi, "AutoRed: Automated attack scenario generation framework for red teaming of LLMs," in *Proc. IEEE Int. Conf. Big Data (BigData)*, Dec. 2024, pp. 2376–2383.
[90] Z. Wang, N. Nagaraja, L. Zhang, H. Bahsi, P. Patil, and P. Liu, "To protect the LLM agent against the prompt injection attack with polymorphic prompt," in *Proc. 55th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.-Supplemental Volume (DSN-S)*, Jun. 2025, pp. 22–28.
[91] Q. Wang, D. Feng, X. Zhang, A. Shen, Y. Xu, B. Ding, and H. Wang, "Pay more attention to the robustness of LLMs on adversarial prompt for instruction data mining," *Neural Netw.*, vol. 193, Jan. 2026, Art. no. 107989.
[92] H. Yang, "TAPDA: Text adversarial purification as defense against adversarial prompt attack for large language models," in *Proc. 8th Int. Conf. Adv. Algorithms Control Eng. (ICAACE)*, Mar. 2025, pp. 1998–2004.
[93] G. Yang, Z. Sun, and Y. Wang, "ShellBox: Adversarially enhanced LLM-interactive honeypot framework," *IEEE Access*, vol. 13, pp. 143618–143630, 2025.
[94] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, "Benchmarking and defending against indirect prompt injection attacks on large language models," in *Proc. 31st ACM SIGKDD Conf. Knowl. Discovery Data Mining V.1*, Jul. 2025, pp. 1809–1820.
[95] X. Yi, L. Wang, X. Wang, and L. He, "Fine-grained detoxification framework via instance-level prefixes for large language models," *Neurocomputing*, vol. 611, pp. 1–16, Jan. 2025.
[96] Z. Ying, A. Liu, T. Zhang, Z. Yu, S. Liang, X. Liu, and D. Tao, "Jailbreak vision language models via bi-modal adversarial prompt," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 7153–7165, 2025.
[97] D. W. Yip, A. Esmradi, and C. F. Chan, "A novel evaluation framework for assessing resilience against prompt injection attacks in large language models," in *Proc. IEEE Asia Pacific Conf. Comput. Sci. Data Eng. (CSDE)*, Dec. 2023, pp. 1–5.
[98] W. Zhang, X. Kong, C. Dewitt, T. Braunl, and J. B. Hong, "A study on prompt injection attack against LLM-integrated mobile robotic systems," in *Proc. IEEE 35th Int. Symp. Softw. Rel. Eng. Workshops (ISSREW)*, Oct. 2024, pp. 361–368.
[99] C. Zhang, M. Jin, Q. Yu, C. Liu, H. Xue, and X. Jin, "Goal-guided generative prompt injection attack on large language models," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Dec. 2024, pp. 941–946.
[100] W. Zhang, X. Kong, C. Dewitt, T. Bräunl, and J. B. Hong, "Enhancing reliability in LLM-integrated robotic systems: A unified approach to security and safety," *J. Syst. Softw.*, vol. 231, Jan. 2026, Art. no. 112614.
[101] K. Zhu, Q. Zhao, H. Chen, J. Wang, and X. Xie, "PromptBench: A unified library for evaluation of large language models," *J. Mach. Learn. Res.*, pp. 12287–12308, Jan. 2023.
[102] T. Dybå and T. Dingsøyr, "Strength of evidence in systematic reviews in software engineering," in *Proc. 2nd ACM-IEEE Int. Symp. Empirical Softw. Eng. Meas.*, Oct. 2008, pp. 178–187.



CHRISTIAN WOESLE (Member, IEEE) received the B.Sc. degree in business informatics from the FOM University of Applied Sciences, Munich, Germany, and the master's degree in systems engineering from the University of Applied Sciences, Munich. He is currently pursuing the Ph.D. degree with the Chair of Hybrid Intelligence, Helmut-Schmidt-University/University of the Federal Armed Forces Hamburg. His current research interests include deep learning, hallucinations in LLMs, multi-agent systems, and natural language processing.



RICARDO BUETTNER (Senior Member, IEEE) received the Dipl.-Inf. degree in computer science and the Dipl.-Wirtsch.-Ing. degree in industrial engineering and management from the Technical University of Ilmenau, Germany, the Dipl.-Kfm. degree in business administration from the University of Hagen, Germany, the Ph.D. degree in information systems from the University of Hohenheim, Germany, and the Habilitation (venia legendi) degree in information systems from the University of Trier, Germany. He is currently a Chaired Professor of hybrid intelligence with the Helmut-Schmidt-University/University of the Federal Armed Forces Hamburg, Germany. He has published more than 180 peer-reviewed articles, including articles in *Electronic Markets*, *AIS Transactions on Human-Computer Interaction*, *Personality and Individual Differences*, *European Journal of Psychological Assessment*, *PLOS One*, and IEEE Access. He has received 21 international best paper, best reviewer, and service awards and award nominations, including best paper awards by *AIS Transactions on Human-Computer Interaction*, *Electronic Markets*, and HICSS, for his work.